Lab 07 - Building Desktop Application Using JavaFX and Hibernate

1. Introduction

Imagine you're an admin of IT department in a University. Your manager has asked you to develop a web application for Library Department to manage the Student, Book (A Student can borrow many books)

BOOKS(id, author, isbn, title)

STUDENTS(id IDENTITY(1,1), email, password, firstName, lastName, marks)

The application has to support adding, viewing, modifying, and removing students—a standardized usage action verbs better known as Create, Read, Update, Delete (CRUD).

This lab explores creating an application using Hibernate, JavaFX. An **SQL Server Database** will be created to persist the student data (reading and managing) by **Hibernate.**

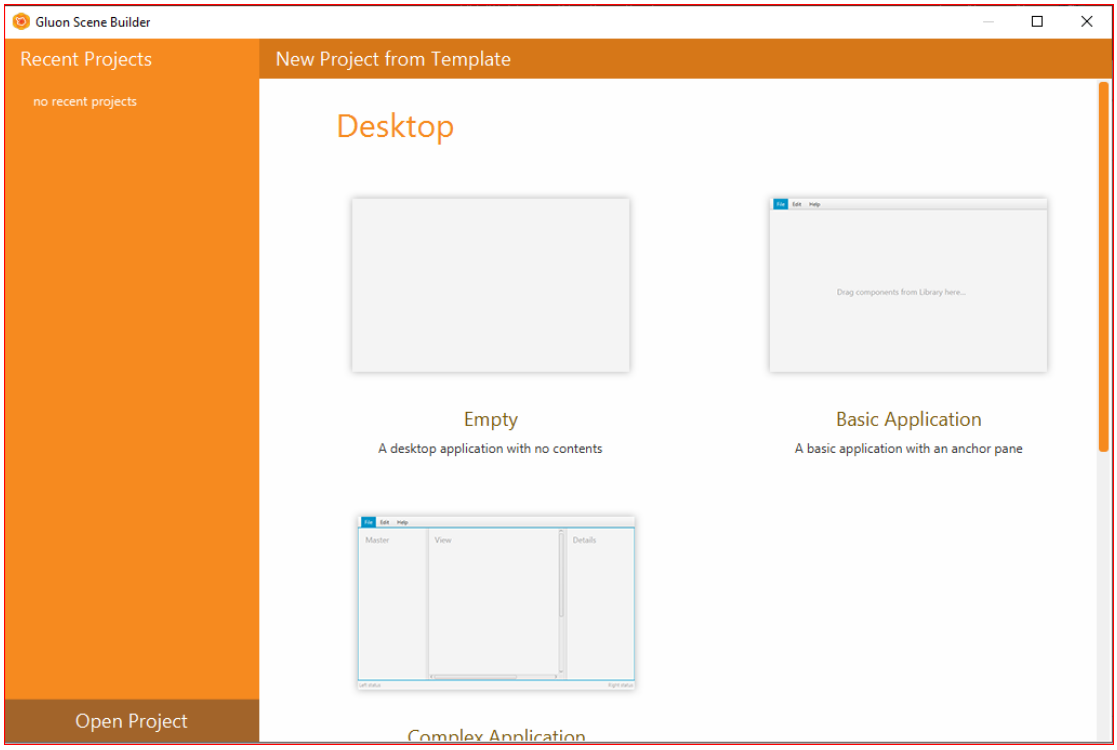
2. Lab Objectives

In this lab, you will:

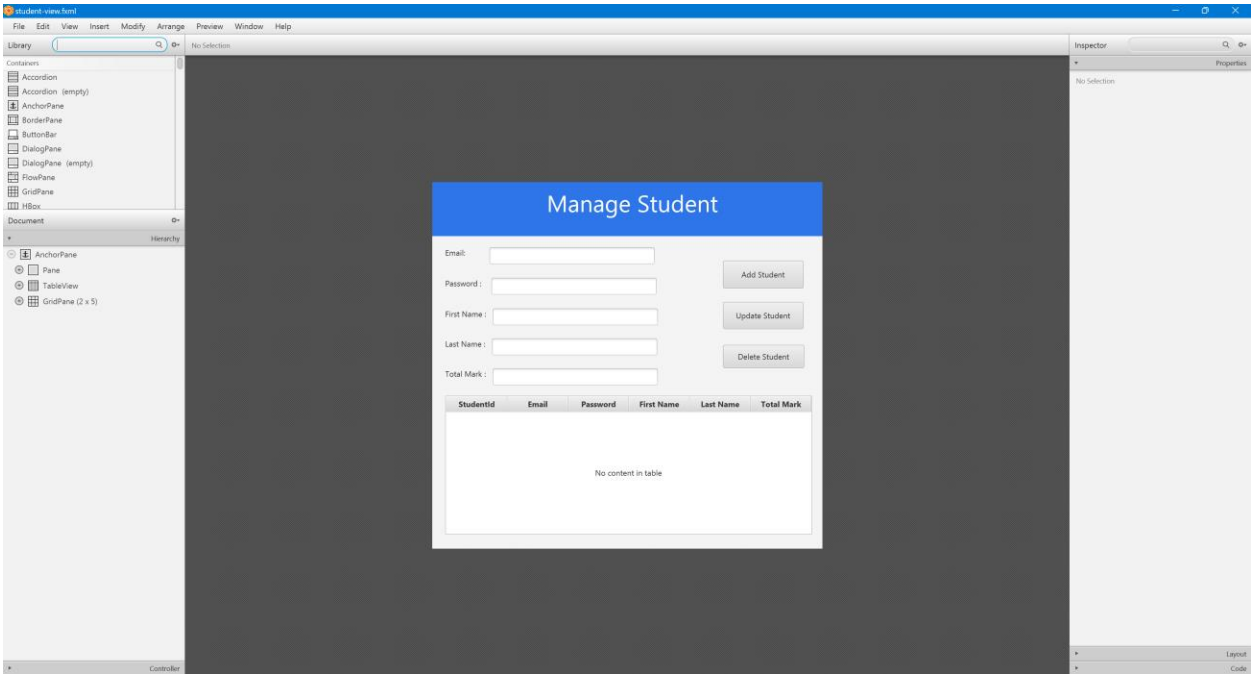
- Use the **IntelliJ**/Eclipse to create Maven Project.
- Develop application using JavaFX, Hibernate
- Use JPA to Create a SQL Server database named LibraryDB that has three tables: STUDENTS, BOOKS, STUDENTS_BOOKS
- Develop 3-Layer with Repository Architecture to perform CRUD actions using Hibernate and JavaFX.
- Run the project and test the application actions.

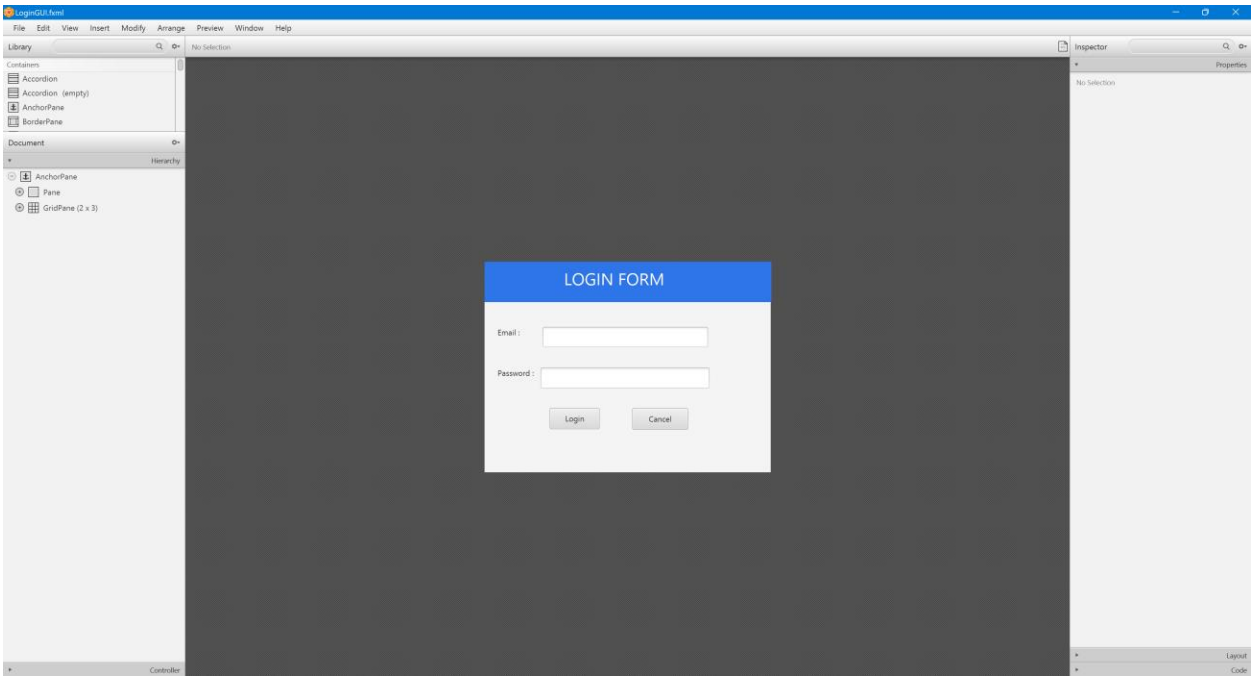
Activity 01: Design the Student Management

Step 01. Open Gluon Scene Builder



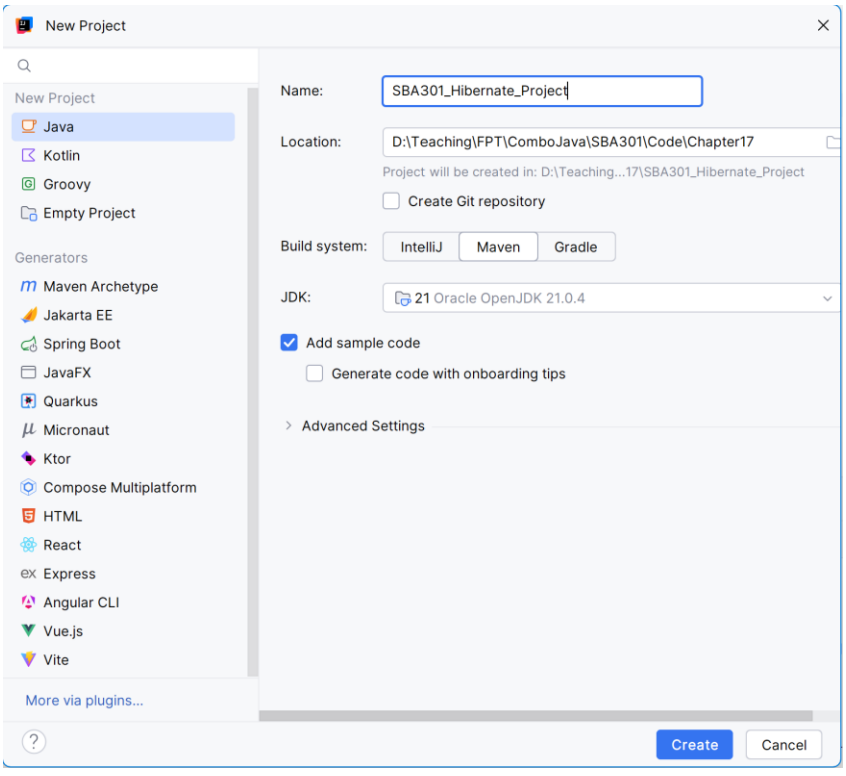
Step 02. Design the Manage Student and Login



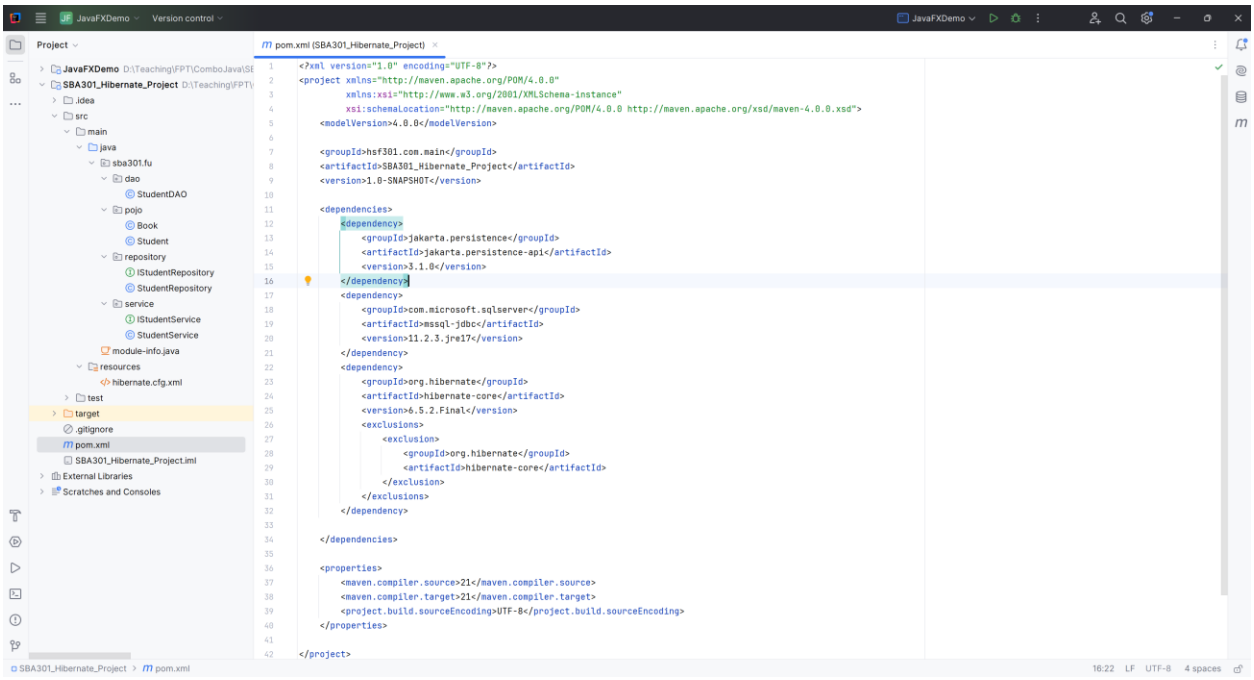


Activity 02: Develop the Architecture with Repository Pattern and show the Student List

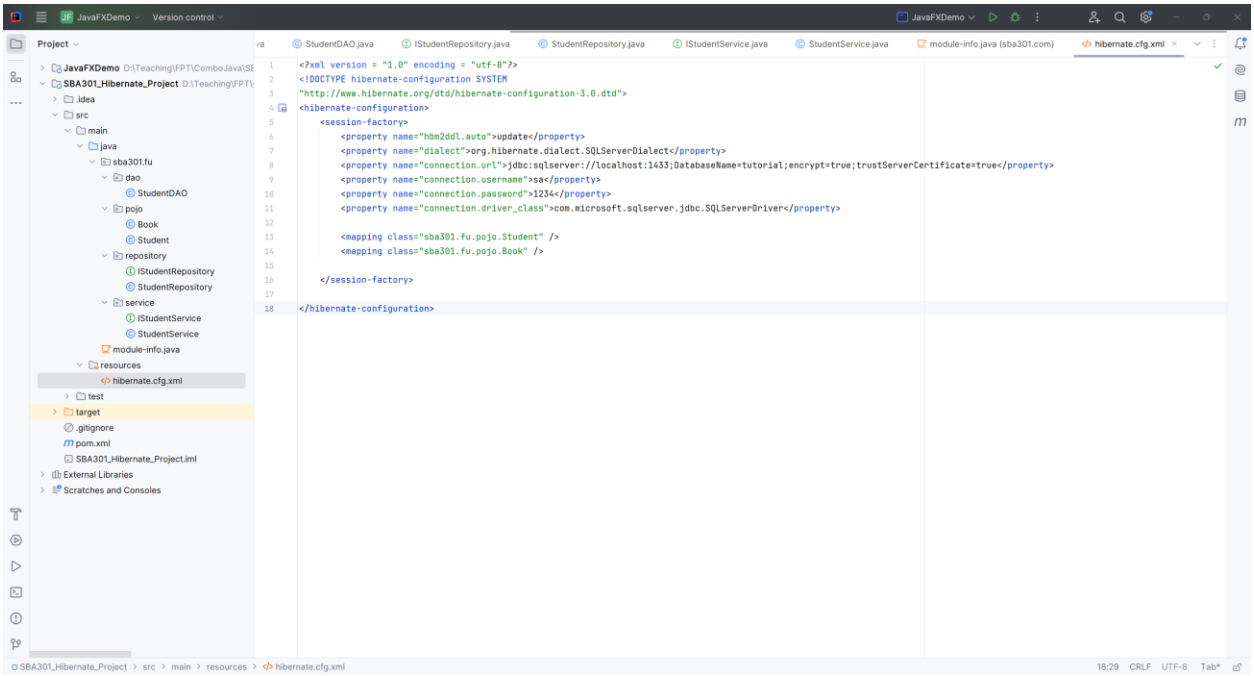
Step 01. Open IntelliJ, File | New | Maven Project



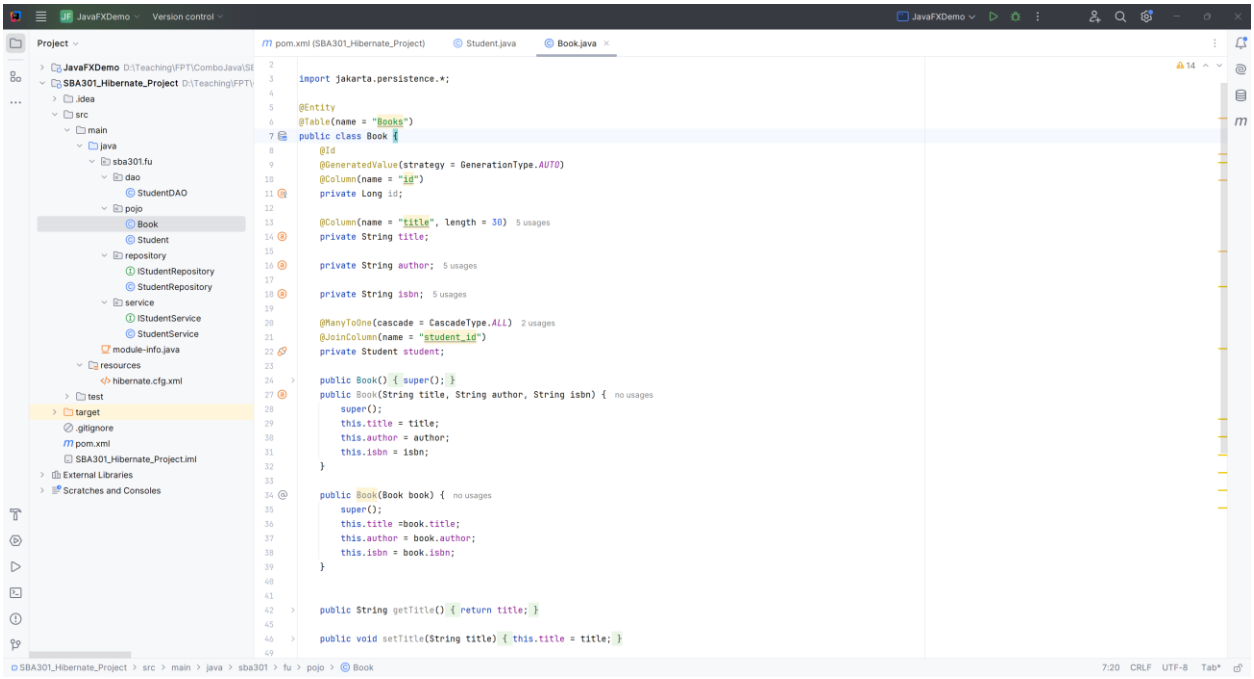
Step 02. Structure of Maven Project

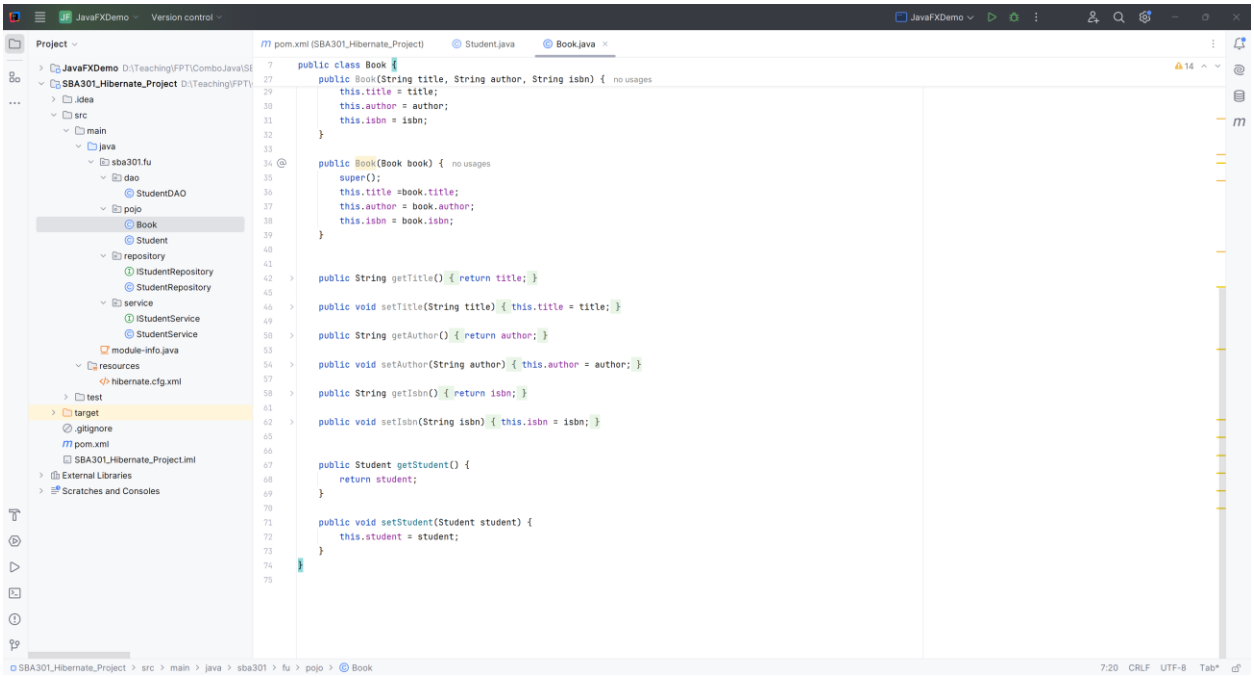


Step 03. Create hibernate.cfg.xml

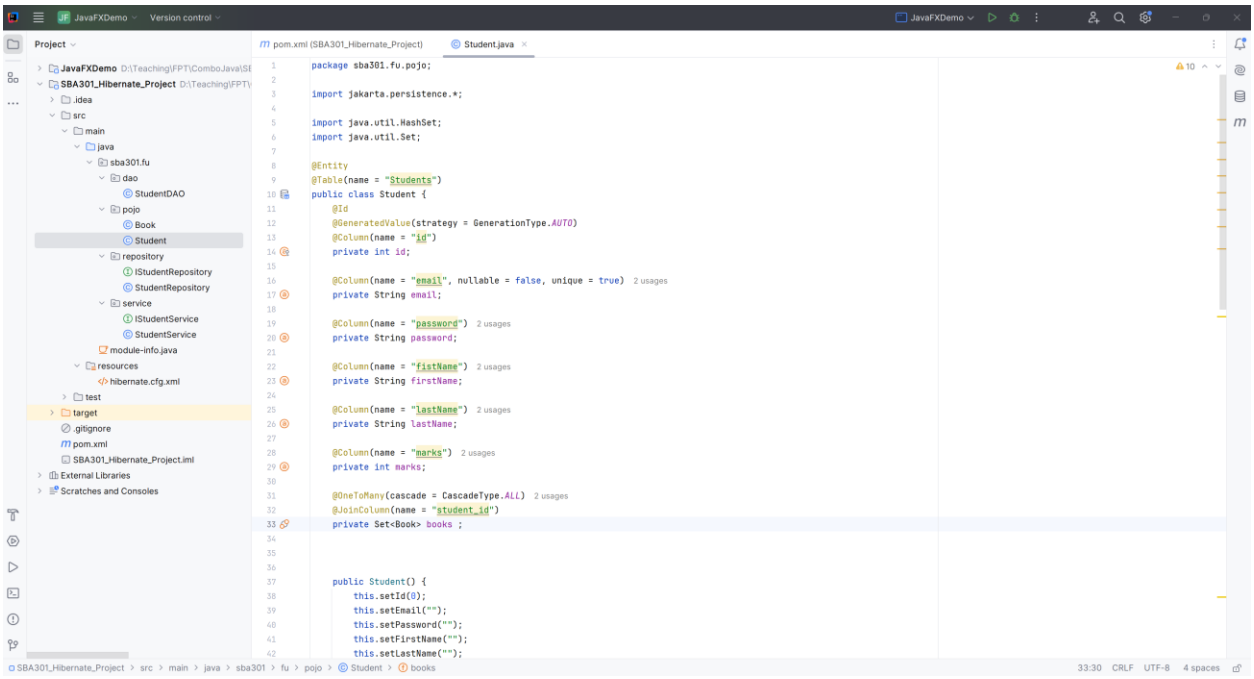


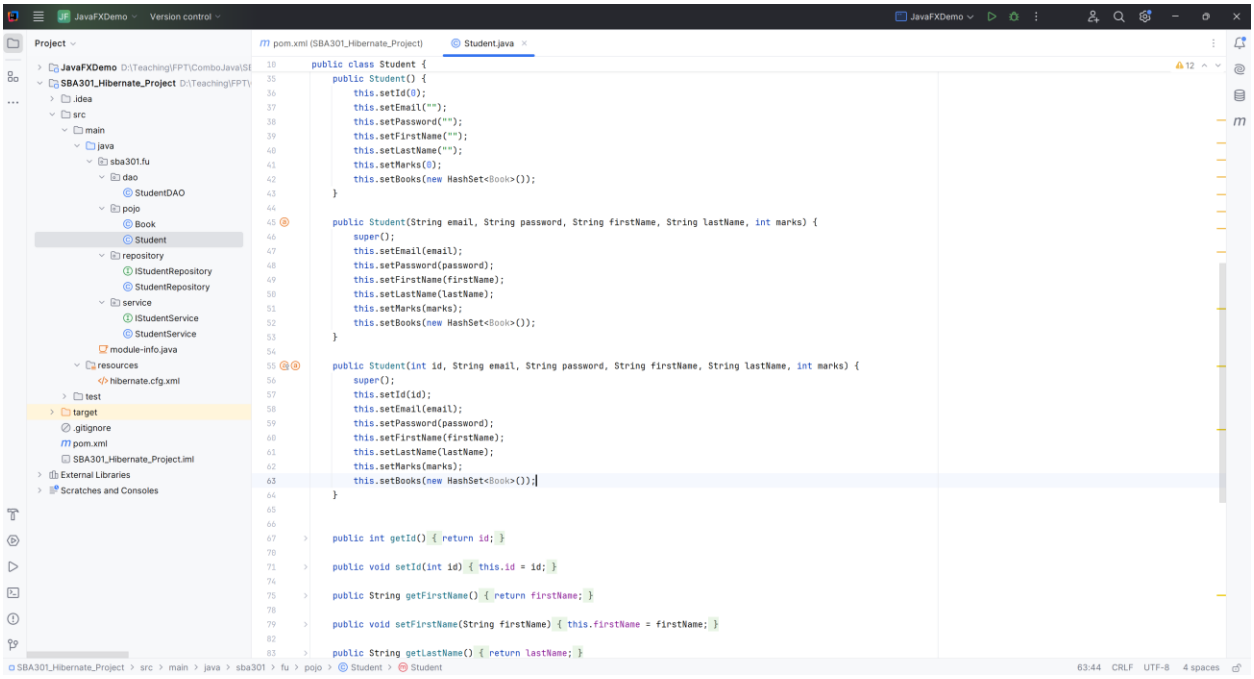
Step 04. Create Books in Pojo



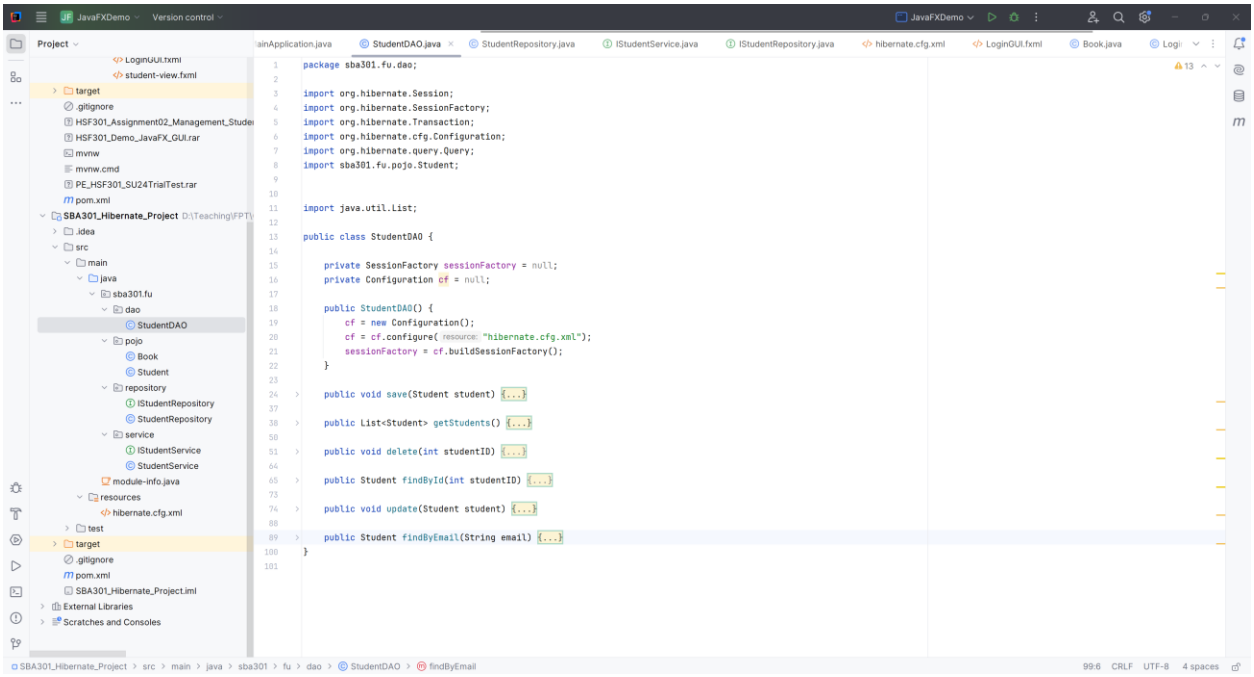


Step 05.Create Student in Pojo



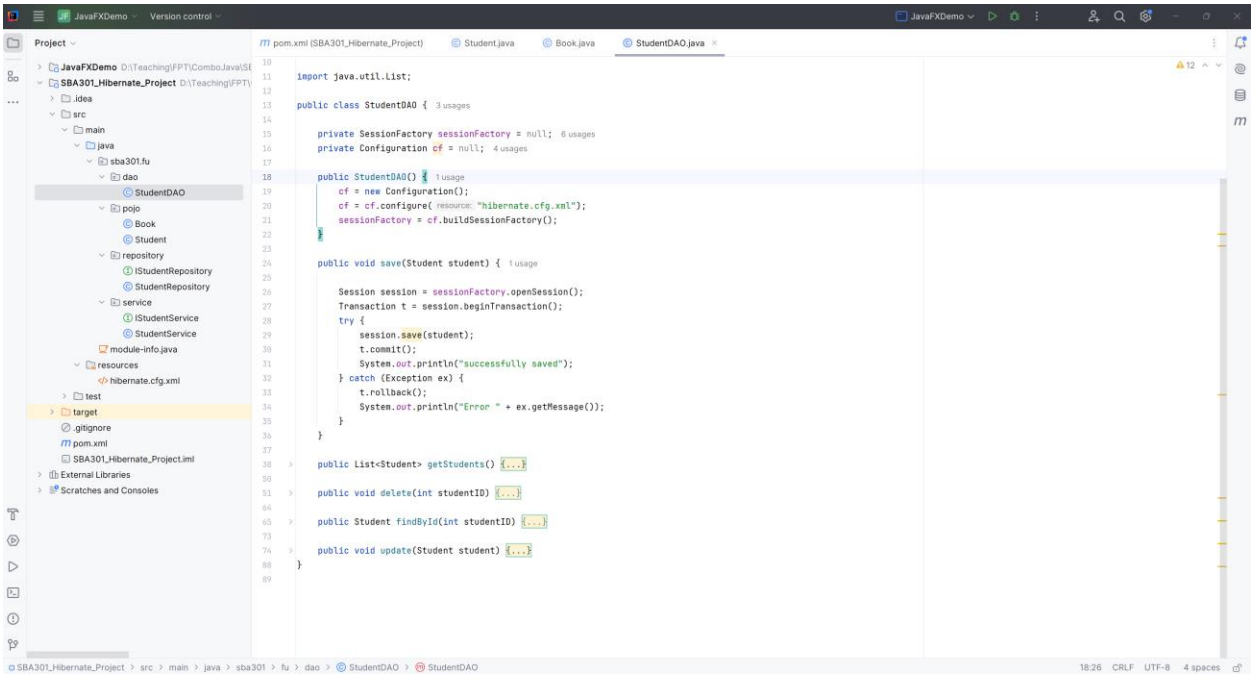


Step 06. Create StudentDAO in DAO package

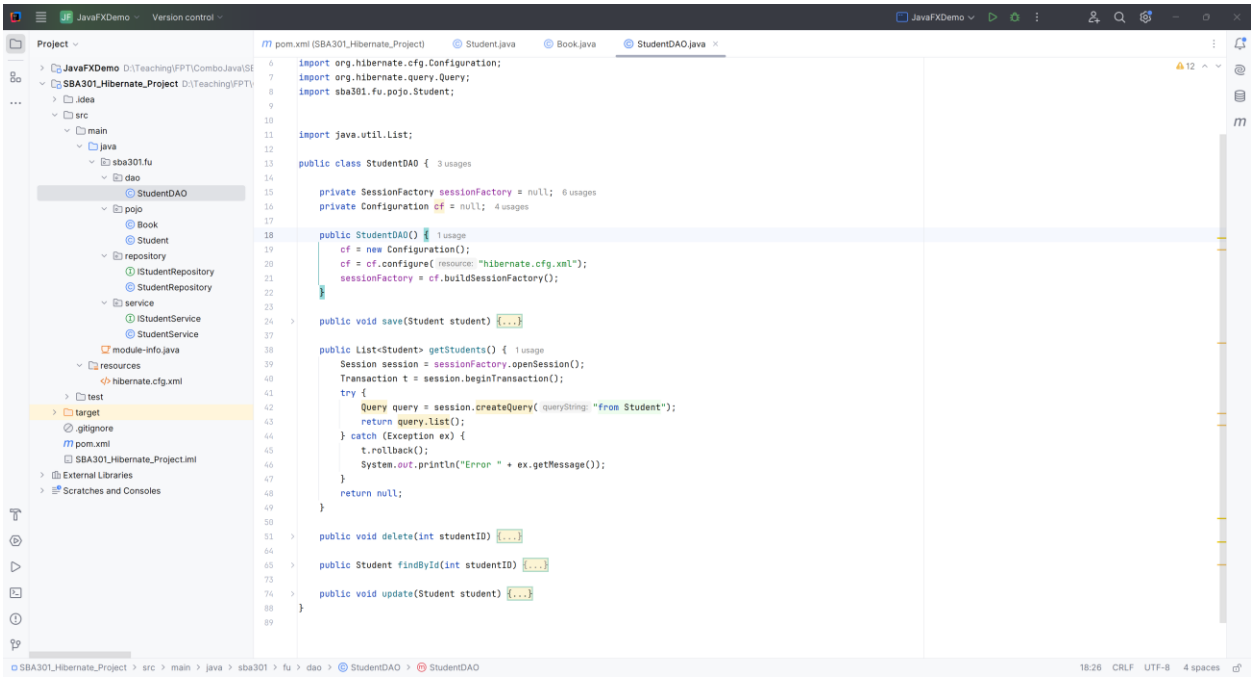


Step 07. CRUD StudentDAO in DAO package

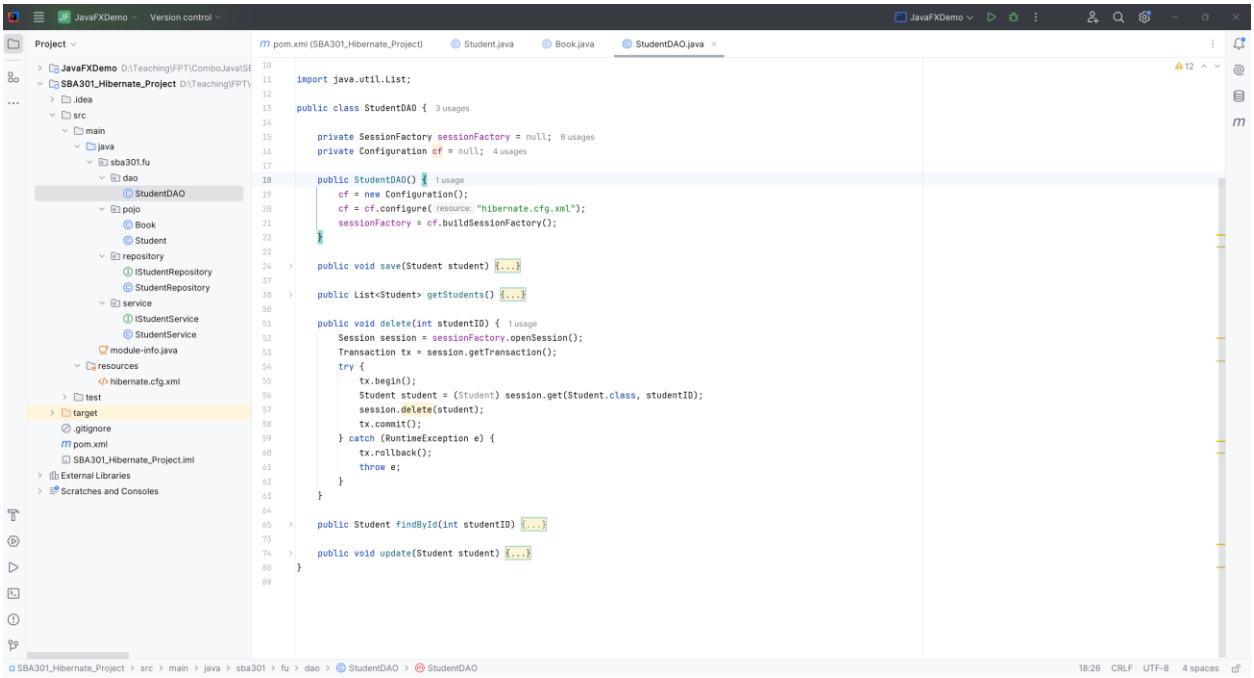
❖ Save Student in StudentDAO



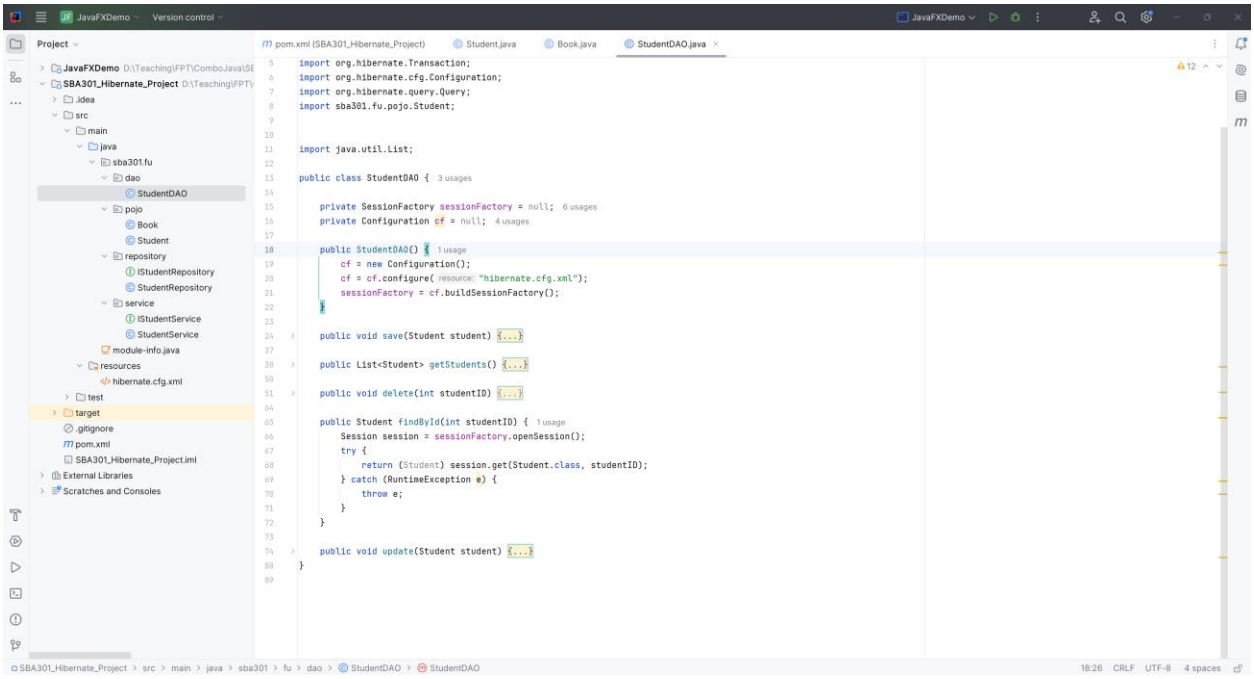
❖ Get All Student in StudentDAO

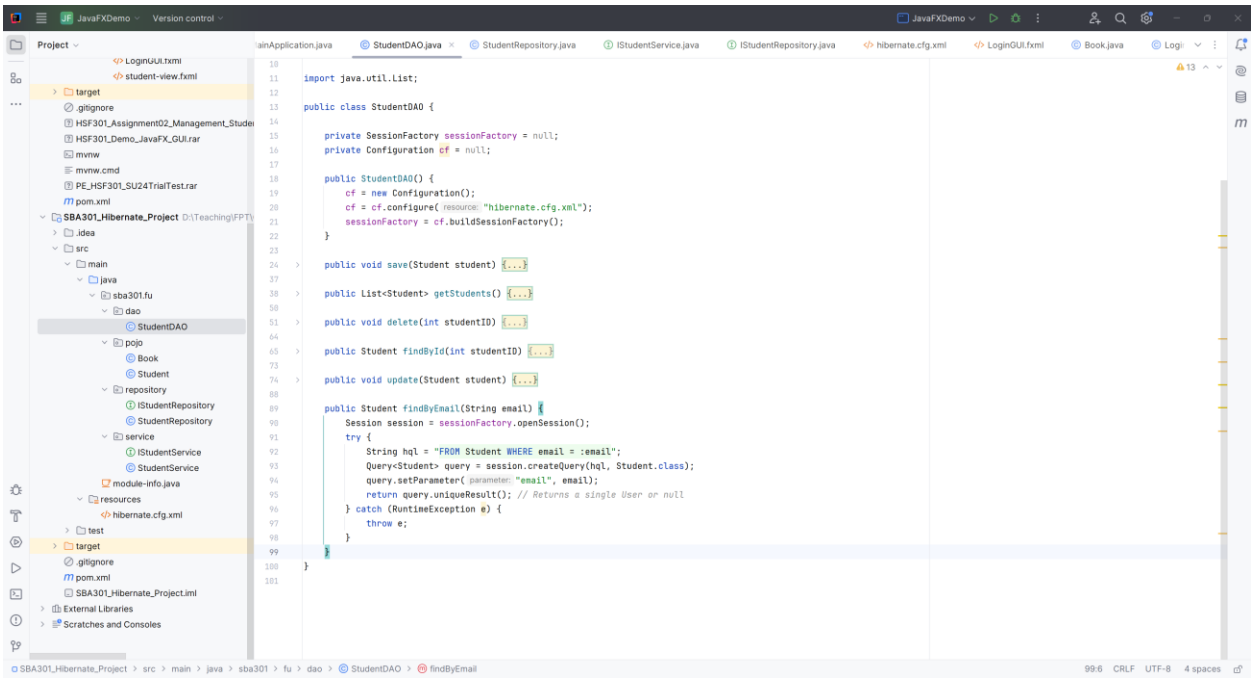


❖ Delete Student in StudentDAO

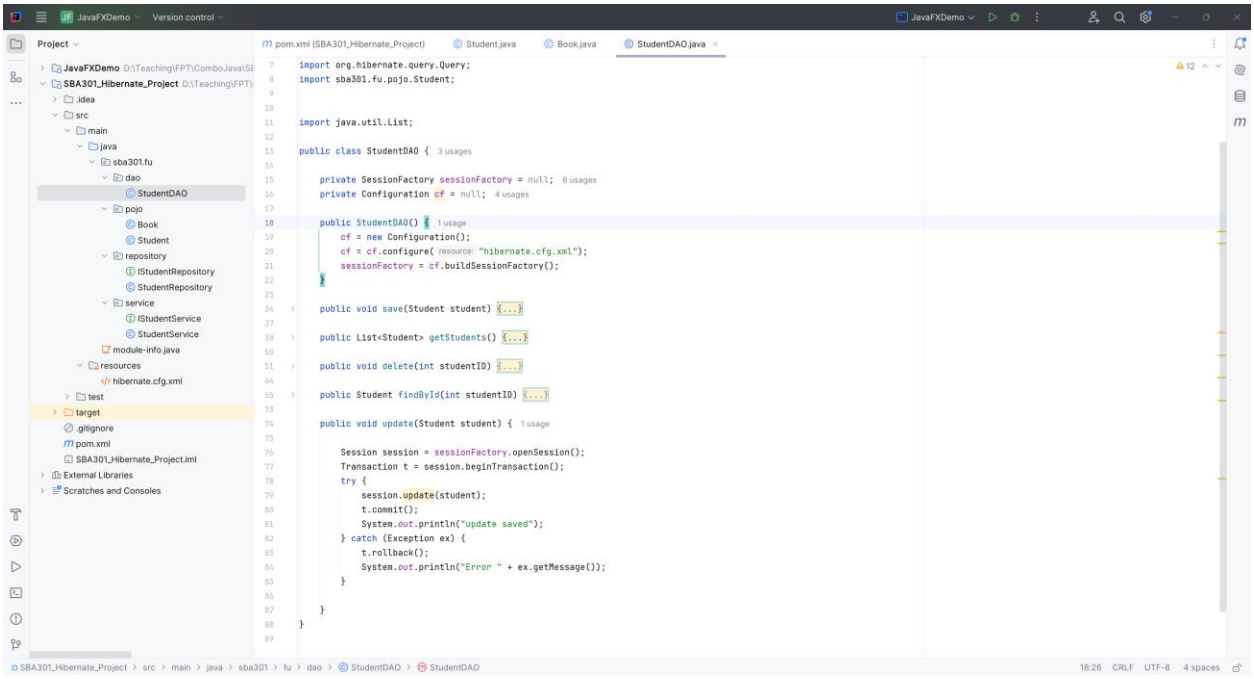


❖ Find A Student in StudentDAO

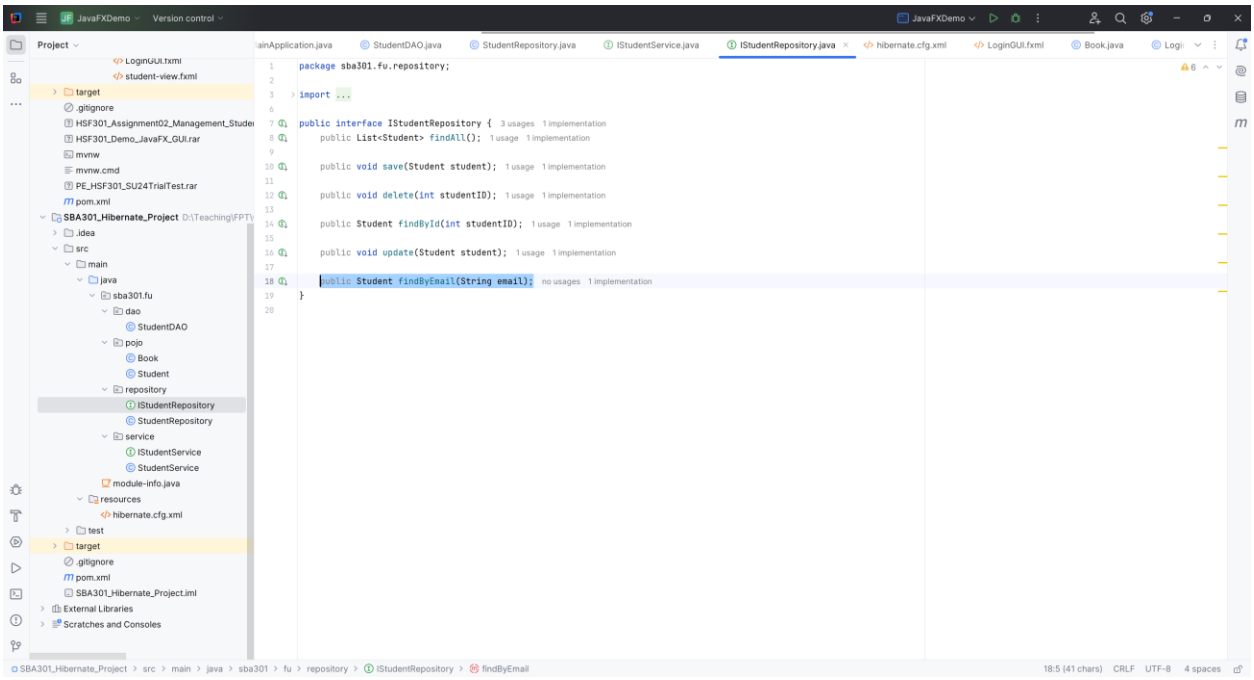




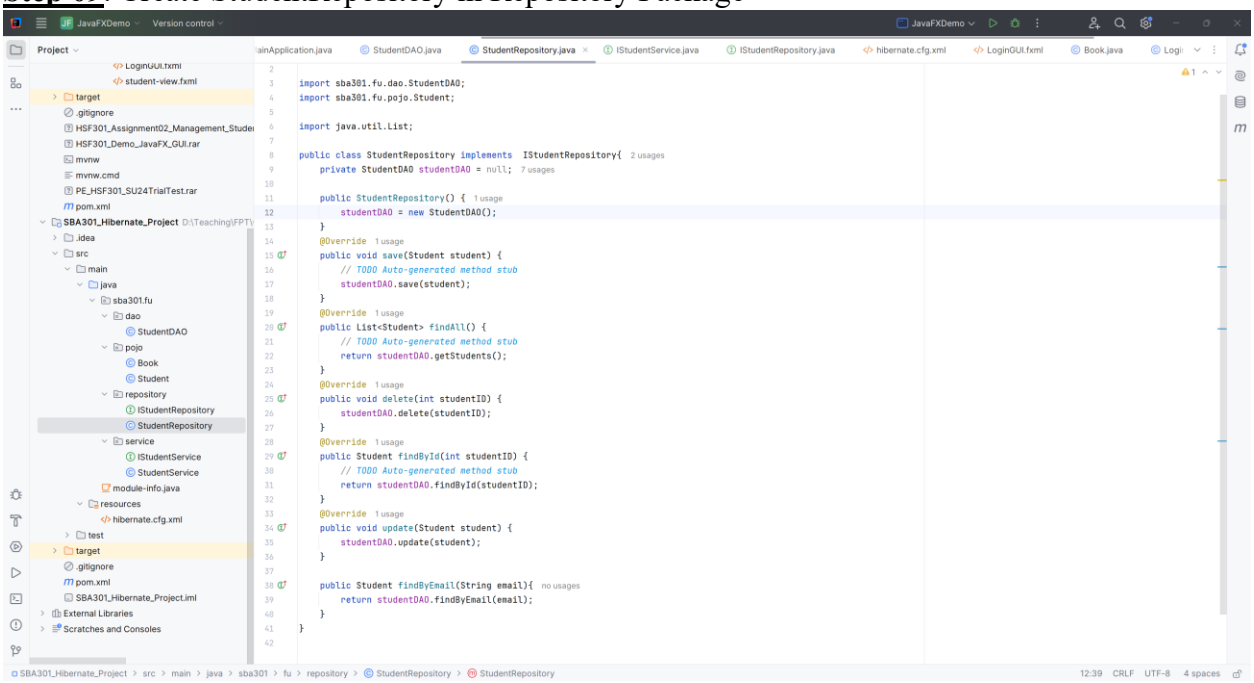
❖ Update a Student in StudentDAO



Step 08. Create IStudentRepository in Repository Package



Step 09. Create StudentRepository in Repository Package



```
import sba301.fu.dao.StudentDAO;
import sba301.fu.pojo.Student;

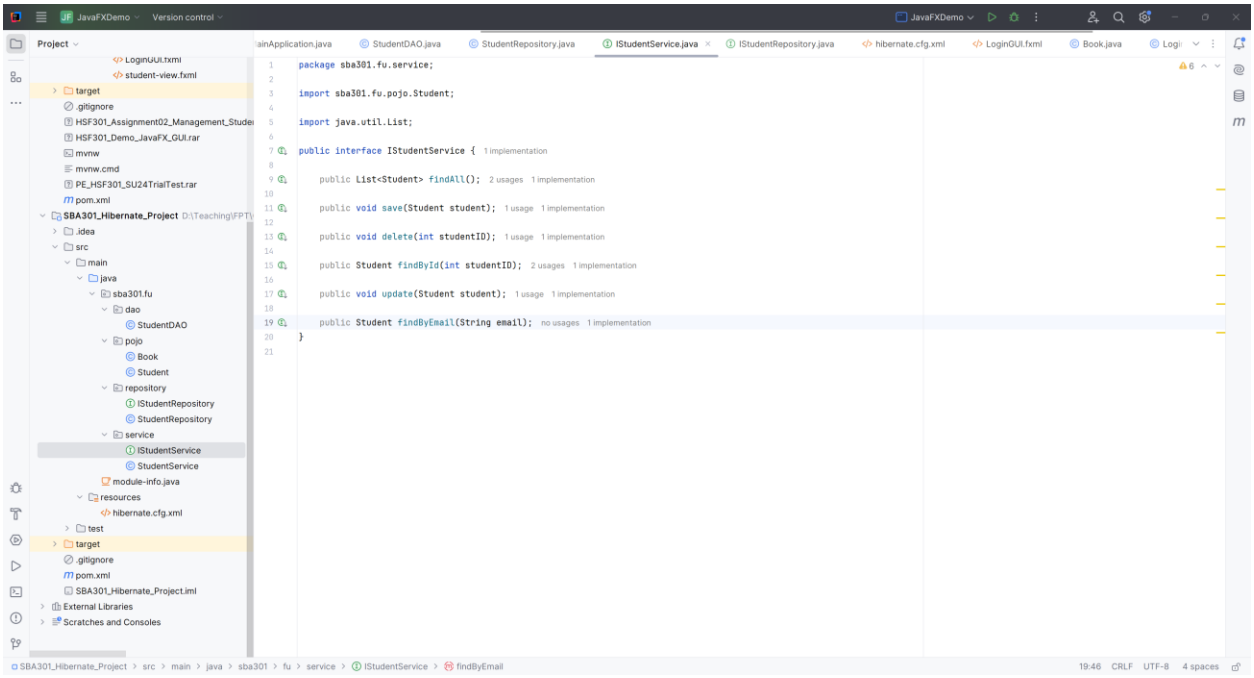
import java.util.List;

public class StudentRepository implements IStudentRepository{ 2 usages
    private StudentDAO studentDAO = null; 7 usages

    public StudentRepository() { 1 usage
        studentDAO = new StudentDAO();
    }
    @Override 1 usage
    public void save(Student student) {
        // TODO Auto-generated method stub
        studentDAO.save(student);
    }
    @Override 1 usage
    public List<Student> findAll() {
        // TODO Auto-generated method stub
        return studentDAO.getStudents();
    }
    @Override 1 usage
    public void delete(int studentID) {
        studentDAO.delete(studentID);
    }
    @Override 1 usage
    public Student findById(int studentID) {
        // TODO Auto-generated method stub
        return studentDAO.findById(studentID);
    }
    @Override 1 usage
    public void update(Student student) {
        studentDAO.update(student);
    }

    public Student findByEmail(String email){ no usages
        return studentDAO.findByEmail(email);
    }
}
```

Step 10. Create IStudentService in Service Package



```
package sba301.fu.service;

import sba301.fu.pojo.Student;

import java.util.List;

public interface IStudentService {  1 implementation

    public List<Student> findAll();  2 usages  1 implementation

    public void save(Student student);  1 usage  1 implementation

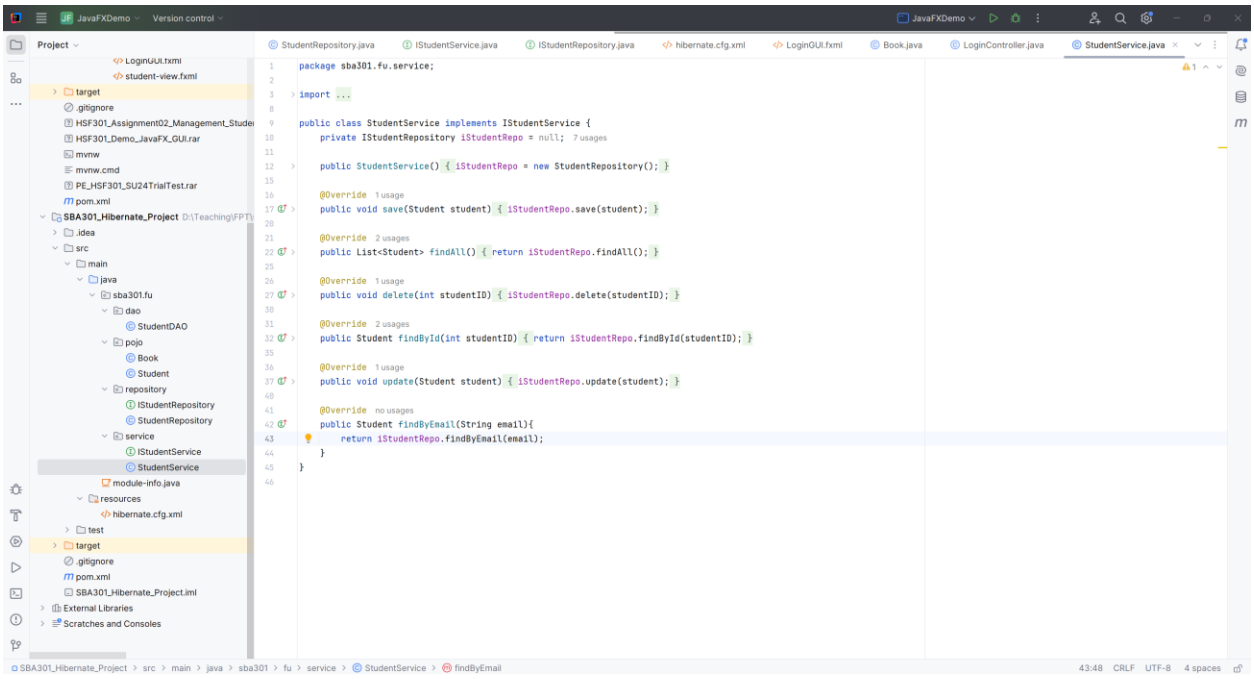
    public void delete(int studentID);  1 usage  1 implementation

    public Student findById(int studentID);  2 usages  1 implementation

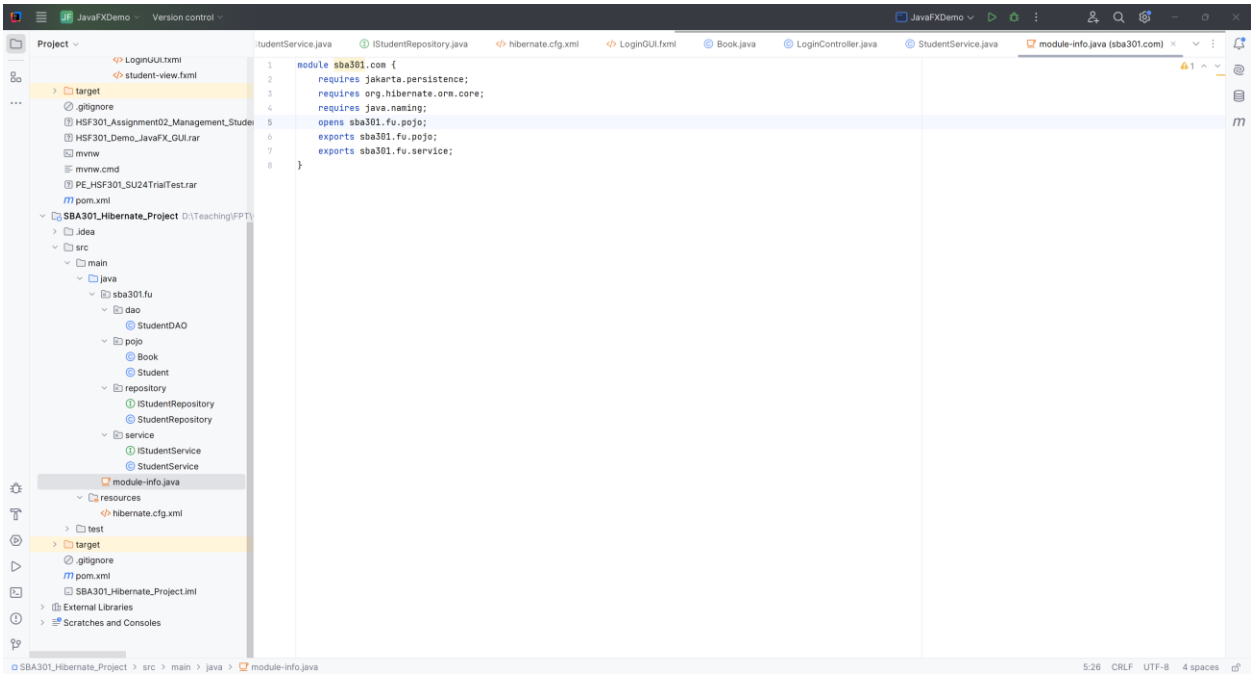
    public void update(Student student);  1 usage  1 implementation

    public Student findByEmail(String email);  no usages  1 implementation
}
```

Step 11. Create StudentService in Service Package

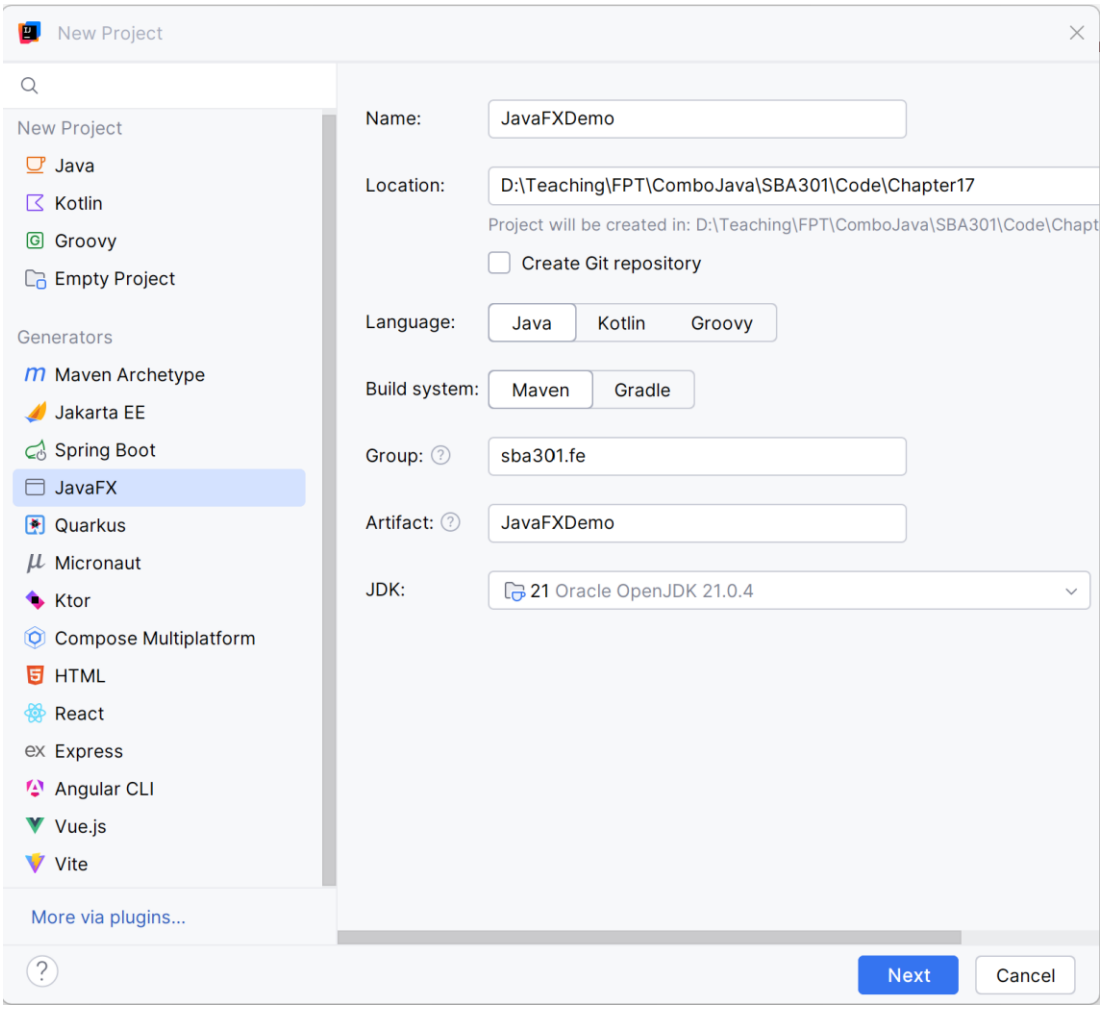


Step 12. Create module-info.java

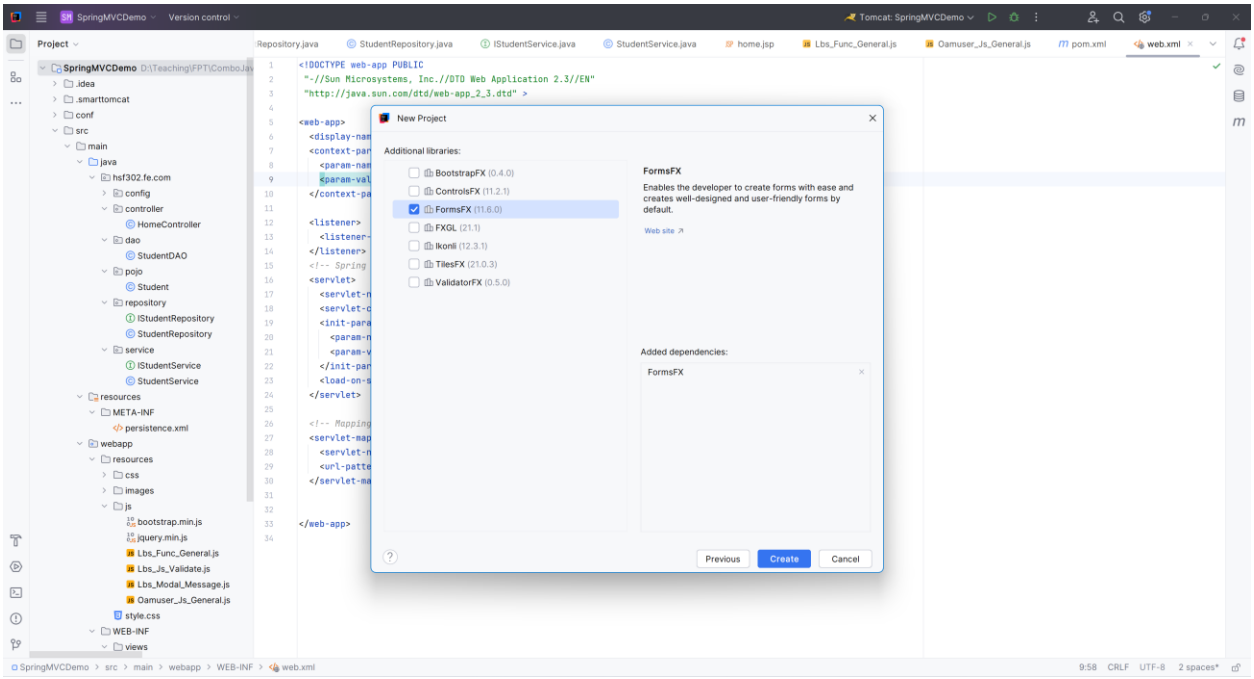


Activity 03: Develop the Student List Page

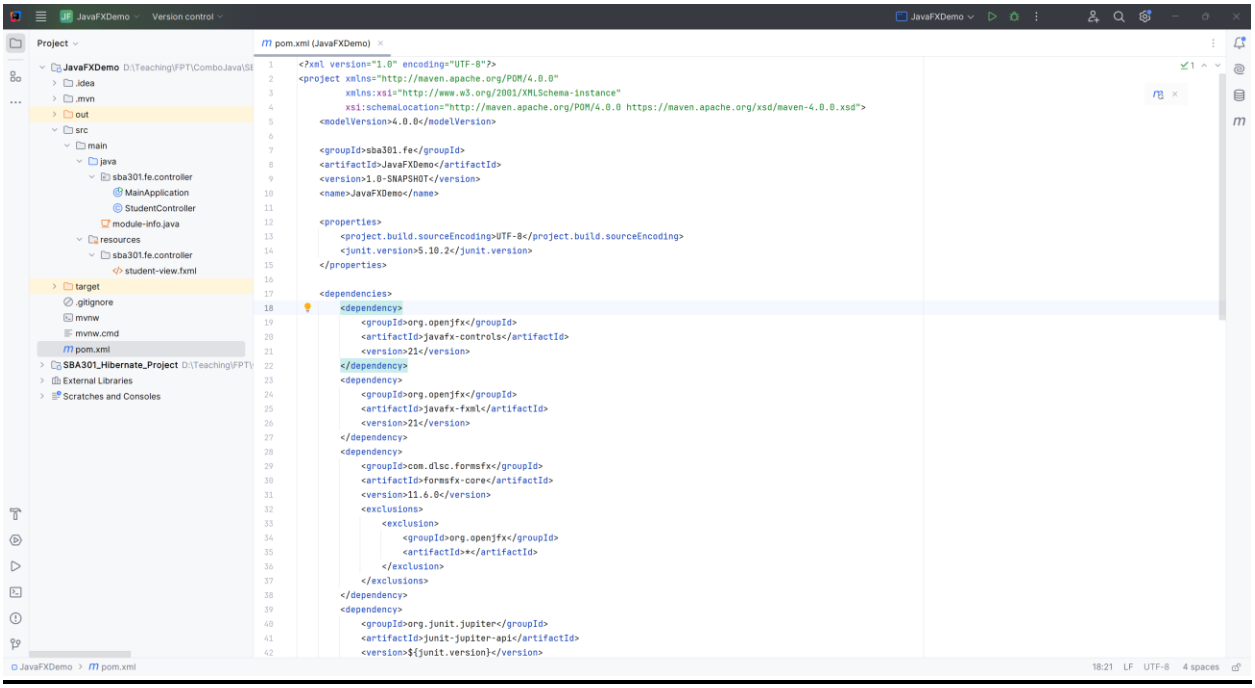
Step 01. Open IntelliJ, File | New | JavaFX Project



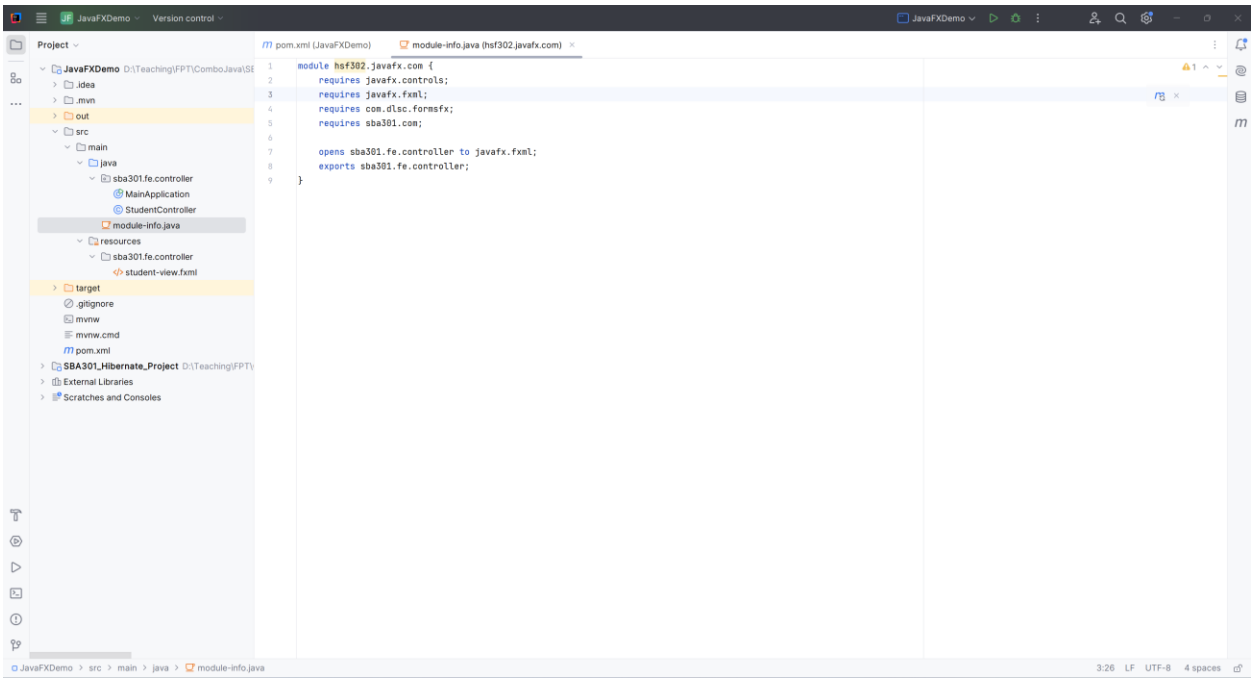
Step 02. Fill the Project Name



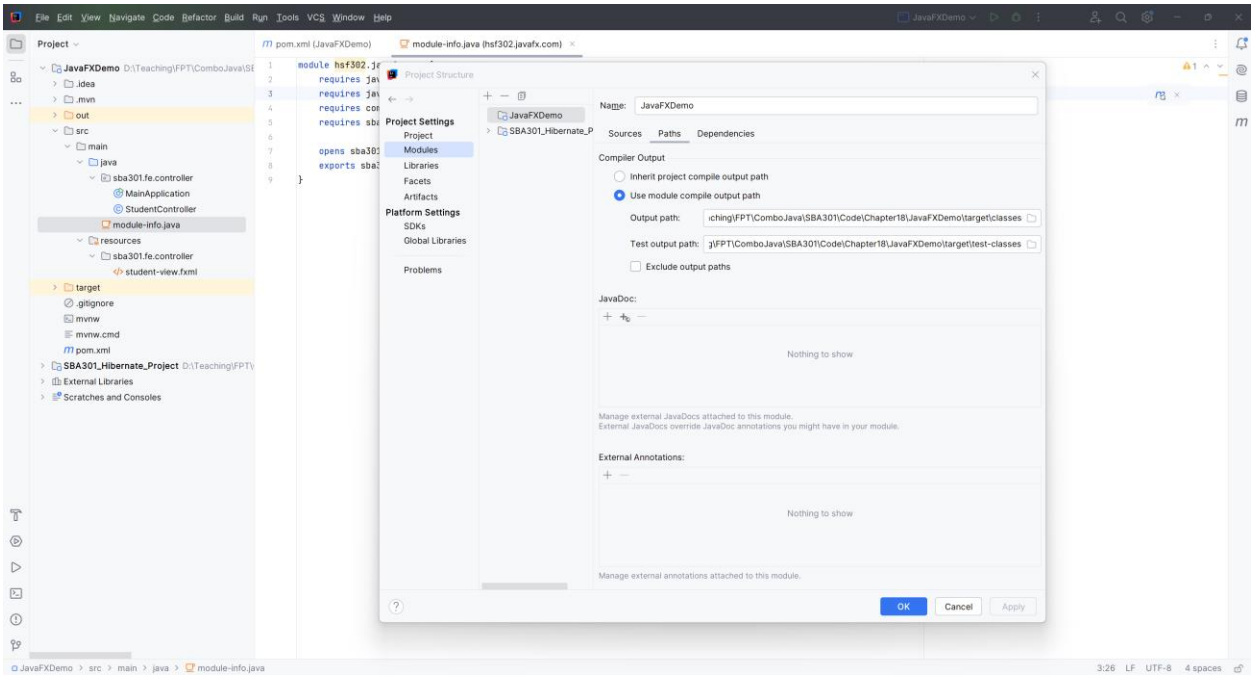
<u>Step 03</u>. Build the Architecture



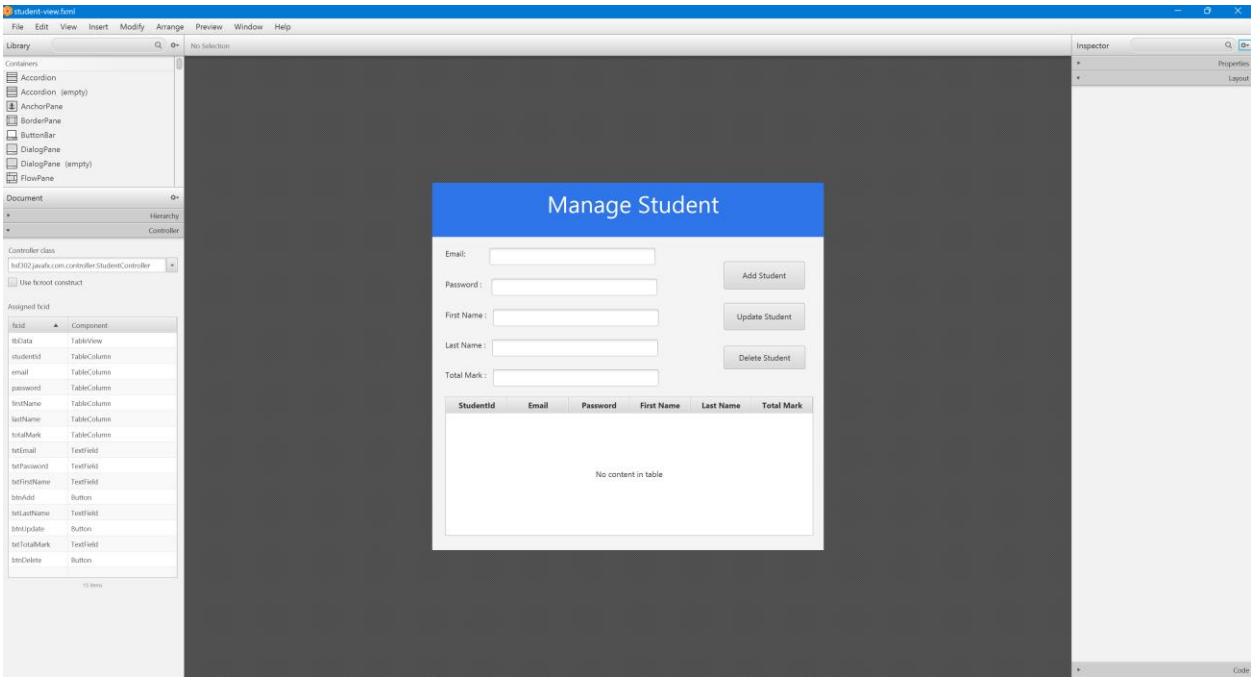
<u>Step 04</u>. Edit module.java in JavaFXDemo



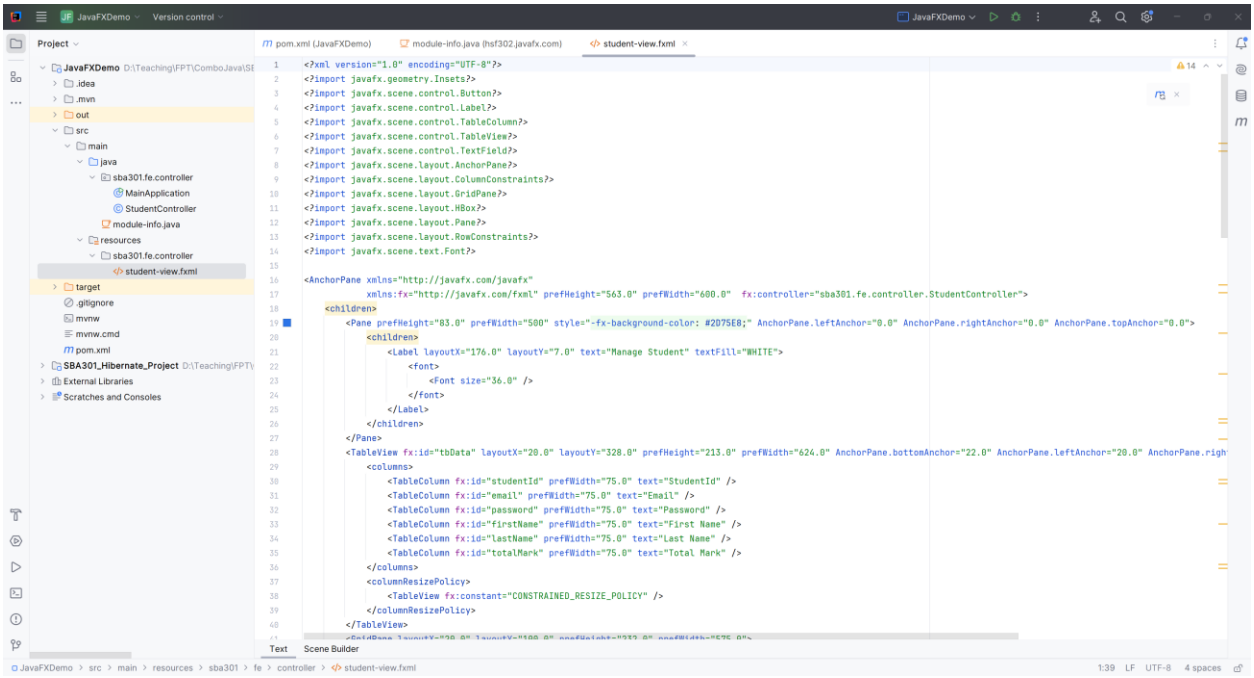
Step 05. Add Reference Another Project

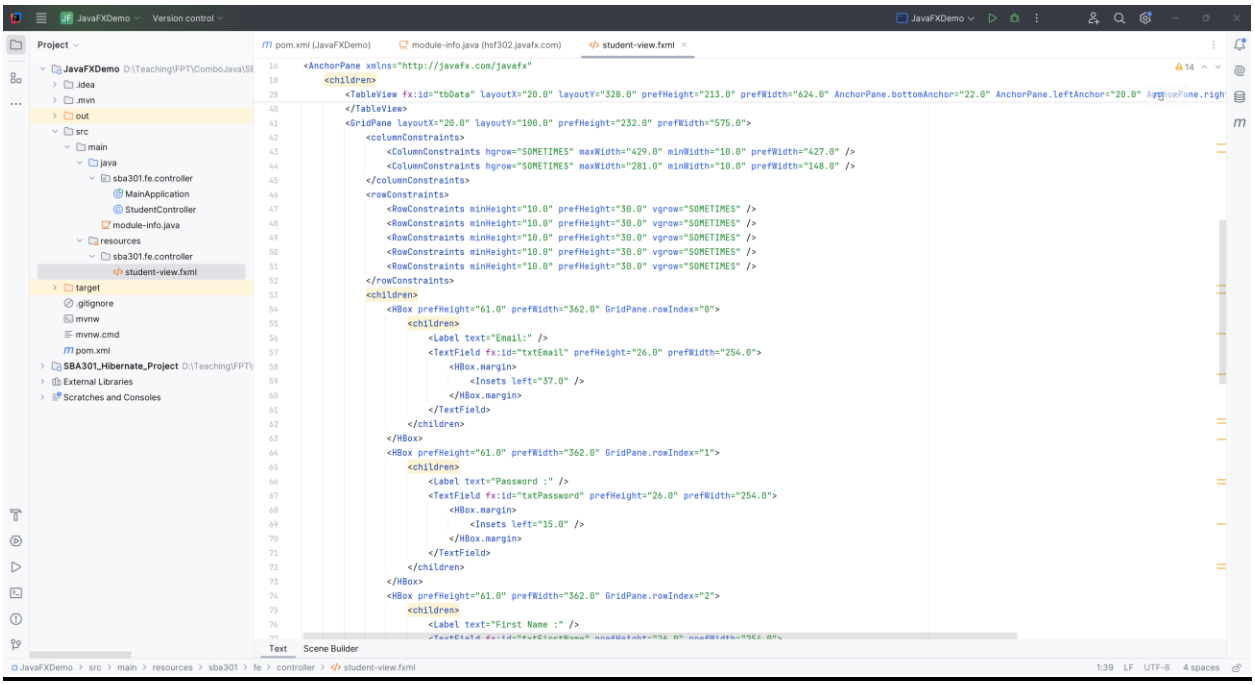


Step 06. Open student-view.fxml in SceneBuilder

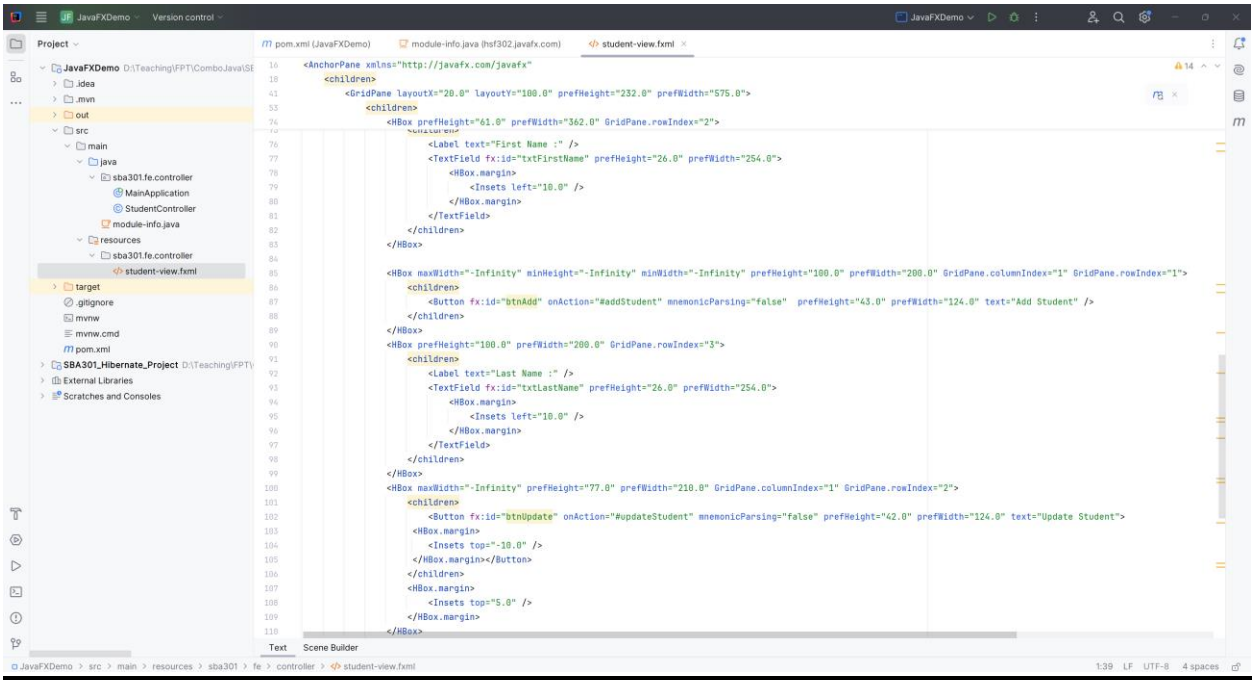


Step 07. Edit student-view.fxml



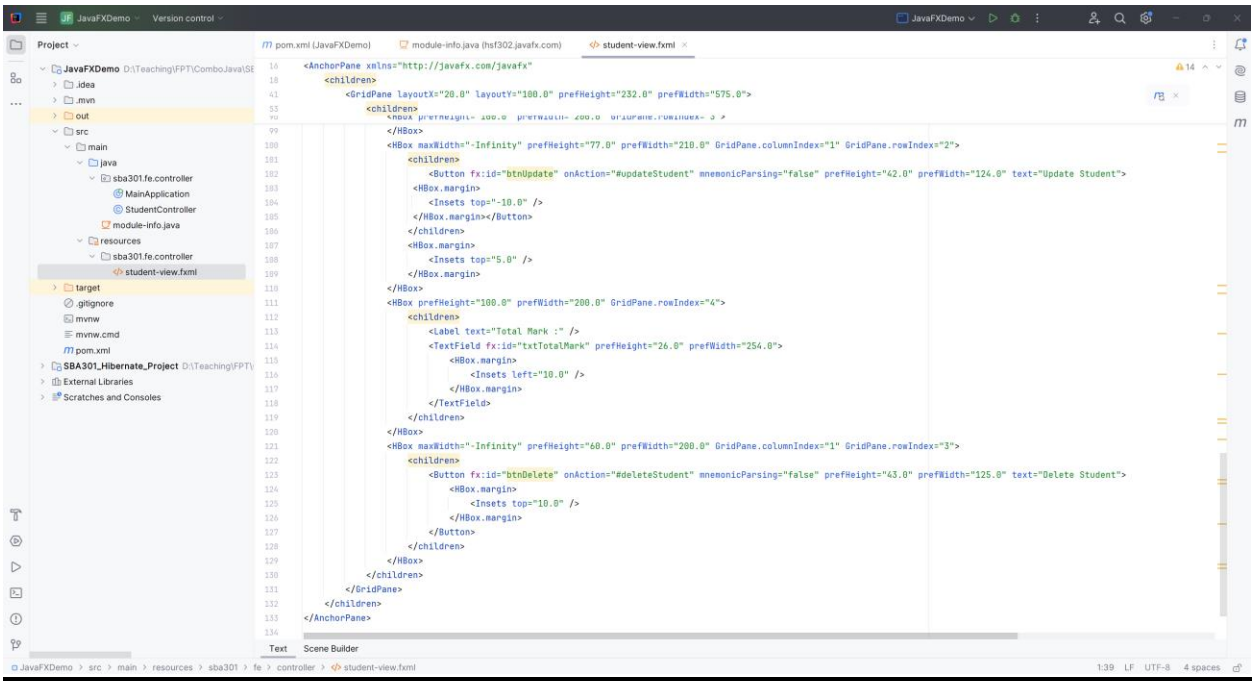


```
<AnchorPane xmlns="http://javafx.com/javafx">
    <children>
        <TableView fx:id="tbData" layoutX="20.0" layoutY="328.0" prefHeight="213.0" prefWidth="624.0" AnchorPane.bottomAnchor="22.0" AnchorPane.leftAnchor="20.0" AnchorPane.righ
        </TableView>
        <GridPane layoutX="20.0" layoutY="100.0" prefHeight="232.0" prefWidth="575.0">
            <columnConstraints>
                <ColumnConstraints hgrow="SOMETIMES" maxWidth="429.0" minWidth="10.0" prefWidth="427.0" />
                <ColumnConstraints hgrow="SOMETIMES" maxWidth="281.0" minWidth="10.0" prefWidth="148.0" />
            </columnConstraints>
            <rowConstraints>
                <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
                <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
                <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
                <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
                <RowConstraints minHeight="10.0" prefHeight="30.0" vgrow="SOMETIMES" />
            </rowConstraints>
            <children>
                <HBox prefHeight="61.0" prefWidth="362.0" GridPane.rowIndex="0">
                    <children>
                        <Label text="Email:" />
                        <TextField fx:id="txtEmail" prefHeight="26.0" prefWidth="254.0">
                            <HBox.margin>
                                <Insets left="37.0" />
                            </HBox.margin>
                        </TextField>
                    </children>
                </HBox>
                <HBox prefHeight="61.0" prefWidth="362.0" GridPane.rowIndex="1">
                    <children>
                        <Label text="Password :" />
                        <TextField fx:id="txtPassword" prefHeight="26.0" prefWidth="254.0">
                            <HBox.margin>
                                <Insets left="15.0" />
                            </HBox.margin>
                        </TextField>
                    </children>
                </HBox>
                <HBox prefHeight="61.0" prefWidth="362.0" GridPane.rowIndex="2">
                    <children>
                        <Label text="First Name :" />
```

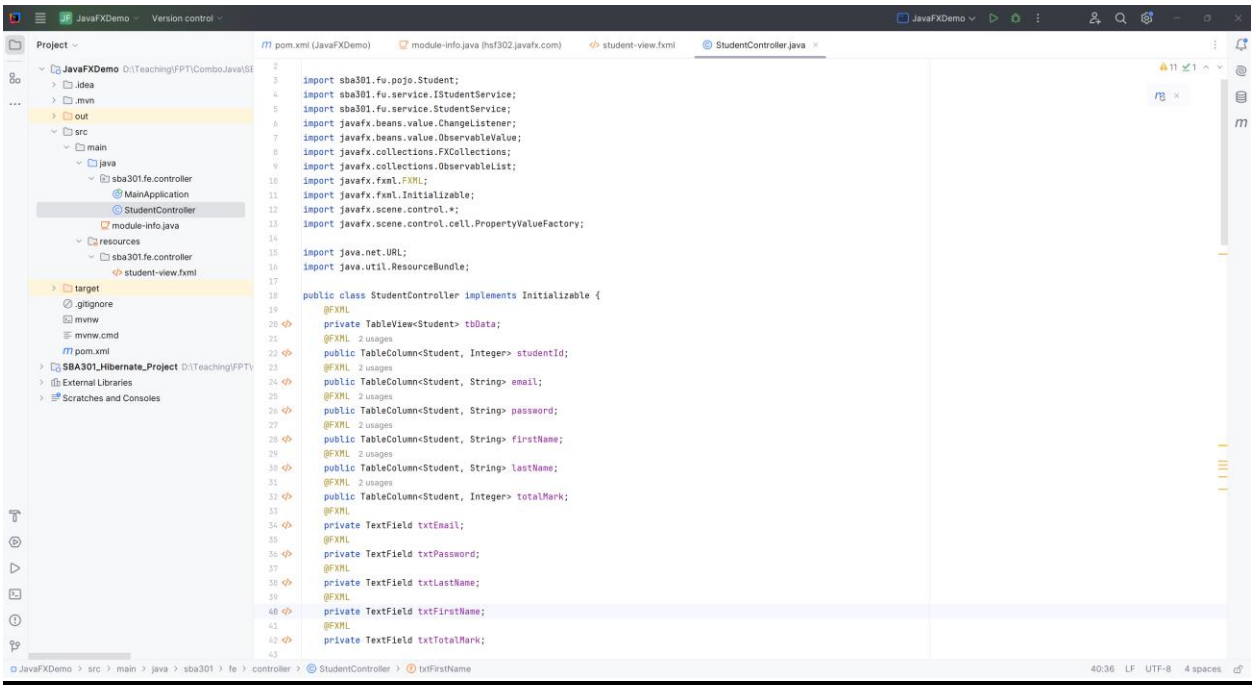


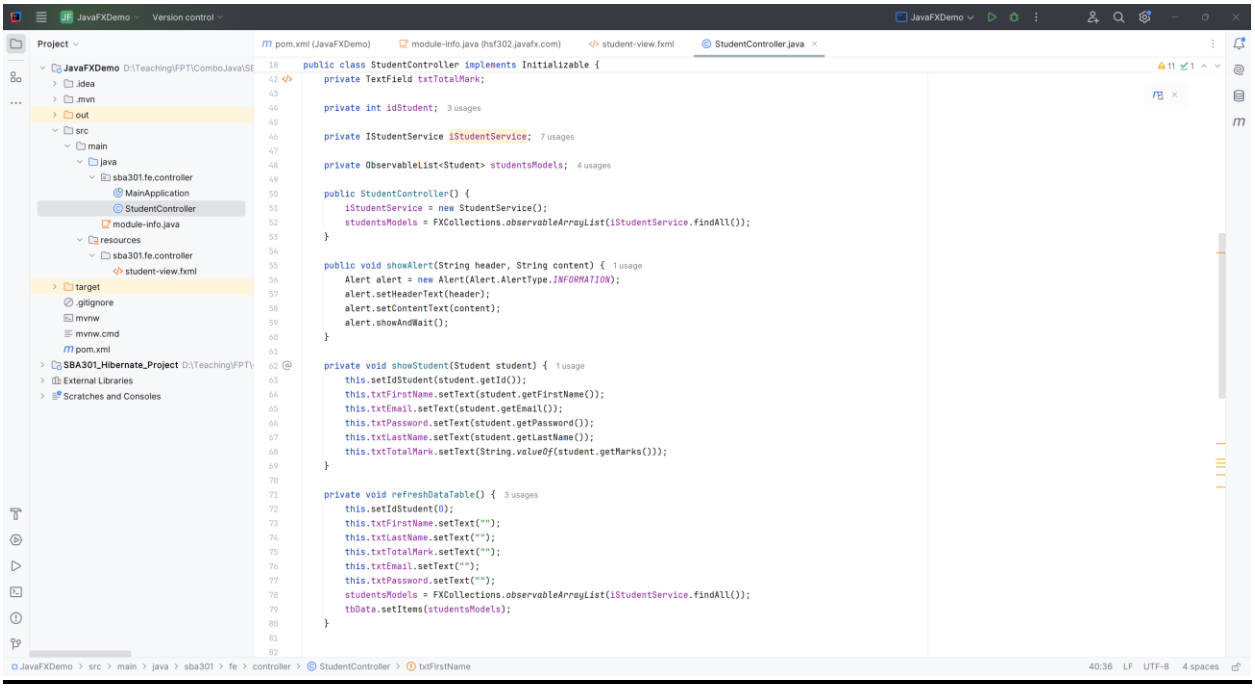
```
<AnchorPane xmlns="http://javafx.com/javafx">
    <children>
        <GridPane layoutX="20.0" layoutY="100.0" prefHeight="232.0" prefWidth="575.0">
            <children>
                <HBox prefHeight="61.0" prefWidth="362.0" GridPane.rowIndex="2">
                    <children>
                        <Label text="First Name :" />
                        <TextField fx:id="txtFirstName" prefHeight="26.0" prefWidth="254.0">
                            <HBox.margin>
                                <Insets left="10.0" />
                            </HBox.margin>
                        </TextField>
                    </children>
                </HBox>

                <HBox maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="100.0" prefWidth="200.0" GridPane.columnIndex="1" GridPane.rowIndex="1">
                    <children>
                        <Button fx:id="btnAdd" onAction="#addStudent" mnemonicParsing="false"  prefHeight="43.0" prefWidth="124.0" text="Add Student" />
                    </children>
                </HBox>
                <HBox prefHeight="100.0" prefWidth="200.0" GridPane.rowIndex="3">
                    <children>
                        <Label text="Last Name :" />
                        <TextField fx:id="txtLastName" prefHeight="26.0" prefWidth="254.0">
                            <HBox.margin>
                                <Insets left="10.0" />
                            </HBox.margin>
                        </TextField>
                    </children>
                </HBox>
                <HBox maxWidth="-Infinity" prefHeight="77.0" prefWidth="210.0" GridPane.columnIndex="1" GridPane.rowIndex="2">
                    <children>
                        <Button fx:id="btnUpdate" onAction="#updateStudent" mnemonicParsing="false" prefHeight="42.0" prefWidth="124.0" text="Update Student">
                     <HBox.margin>
                        <Insets top="-10.0" />
                     </HBox.margin></Button>
                    </children>
                    <HBox.margin>
                        <Insets top="5.0" />
                    </HBox.margin>
                </HBox>
```



Step 08. Edit the StudentController.java





```
public class StudentController implements Initializable {
    private TextField txtTotalMark;

    private int idStudent;  3 usages

    private IStudentService iStudentService;  7 usages

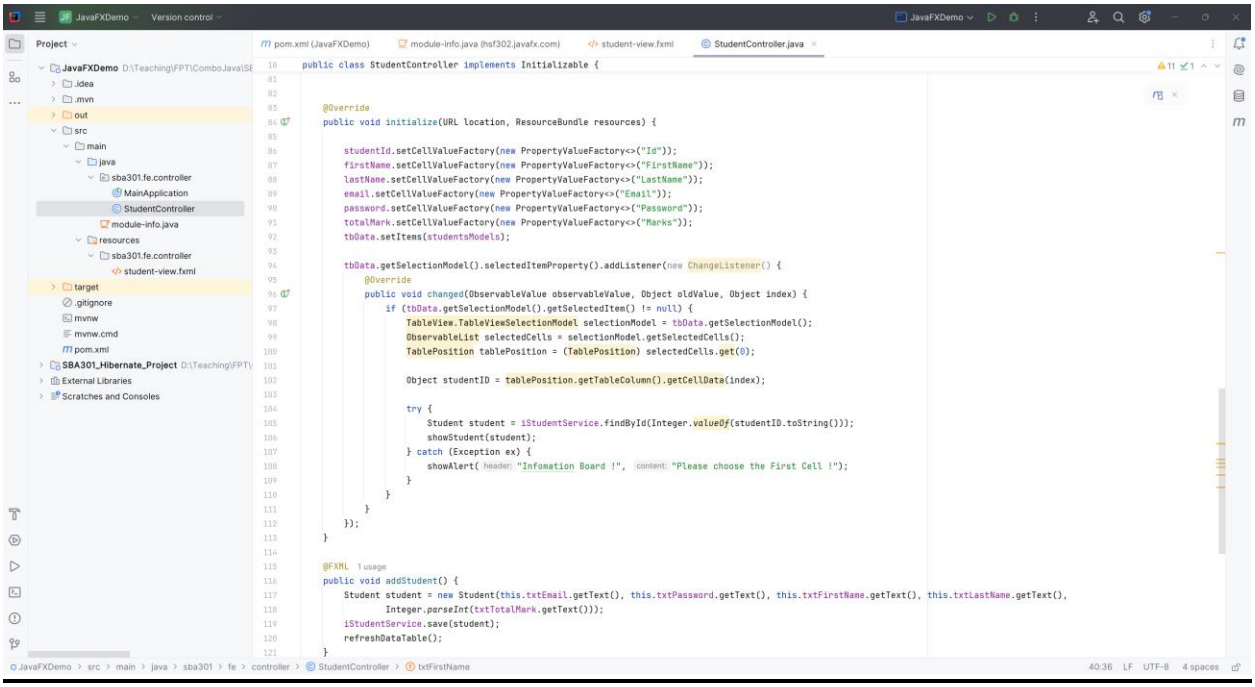
    private ObservableList<Student> studentsModels;  4 usages

    public StudentController() {
        iStudentService = new StudentService();
        studentsModels = FXCollections.observableArrayList(iStudentService.findAll());
    }

    public void showAlert(String header, String content) {  1 usage
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setHeaderText(header);
        alert.setContentText(content);
        alert.showAndWait();
    }

    private void showStudent(Student student) {  1 usage
        this.setIdStudent(student.getId());
        this.txtFirstName.setText(student.getFirstName());
        this.txtEmail.setText(student.getEmail());
        this.txtPassword.setText(student.getPassword());
        this.txtLastName.setText(student.getLastName());
        this.txtTotalMark.setText(String.valueOf(student.getMarks()));
    }

    private void refreshDataTable() {  3 usages
        this.setIdStudent(0);
        this.txtFirstName.setText("");
        this.txtLastName.setText("");
        this.txtTotalMark.setText("");
        this.txtEmail.setText("");
        this.txtPassword.setText("");
        studentsModels = FXCollections.observableArrayList(iStudentService.findAll());
        tbData.setItems(studentsModels);
    }
```



```
public class StudentController implements Initializable {

    @Override
    public void initialize(URL location, ResourceBundle resources) {

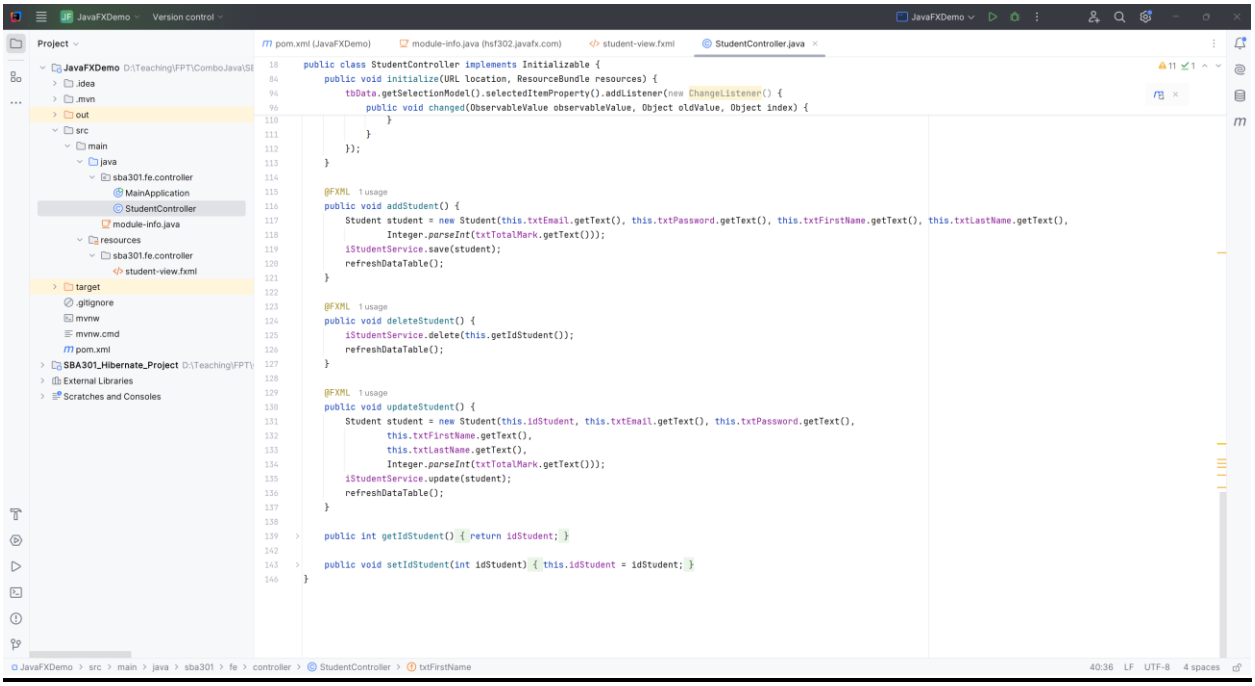
        studentId.setCellValueFactory(new PropertyValueFactory<>("Id"));
        firstName.setCellValueFactory(new PropertyValueFactory<>("FirstName"));
        lastName.setCellValueFactory(new PropertyValueFactory<>("LastName"));
        email.setCellValueFactory(new PropertyValueFactory<>("Email"));
        password.setCellValueFactory(new PropertyValueFactory<>("Password"));
        totalMark.setCellValueFactory(new PropertyValueFactory<>("Marks"));
        tbData.setItems(studentsModels);

        tbData.getSelectionModel().selectedItemProperty().addListener(new ChangeListener() {
            @Override
            public void changed(ObservableValue observableValue, Object oldValue, Object index) {
                if (tbData.getSelectionModel().getSelectedItem() != null) {
                    TableView.TableViewSelectionModel selectionModel = tbData.getSelectionModel();
                    ObservableList selectedCells = selectionModel.getSelectedCells();
                    TablePosition tablePosition = (TablePosition) selectedCells.get(0);

                    Object studentID = tablePosition.getTableColumn().getCellData(index);

                    try {
                        Student student = iStudentService.findById(Integer.valueOf(studentID.toString()));
                        showStudent(student);
                    } catch (Exception ex) {
                        showAlert( header: "Infomation Board !",  content: "Please choose the First Cell !");
                    }
                }
            }
        });
    }

    @FXML  1 usage
    public void addStudent() {
        Student student = new Student(this.txtEmail.getText(), this.txtPassword.getText(), this.txtFirstName.getText(), this.txtLastName.getText(),
                Integer.parseInt(txtTotalMark.getText()));
        iStudentService.save(student);
        refreshDataTable();
    }
```



```
public class StudentController implements Initializable {
    public void initialize(URL location, ResourceBundle resources) {
        tbData.getSelectionModel().selectedItemProperty().addListener(new ChangeListener() {
            public void changed(ObservableValue observableValue, Object oldValue, Object index) {
                }
            }
        });
    }

    @FXML 1 usage
    public void addStudent() {
        Student student = new Student(this.txtEmail.getText(), this.txtPassword.getText(), this.txtFirstName.getText(), this.txtLastName.getText(),
                Integer.parseInt(txtTotalMark.getText()));
        iStudentService.save(student);
        refreshDataTable();
    }

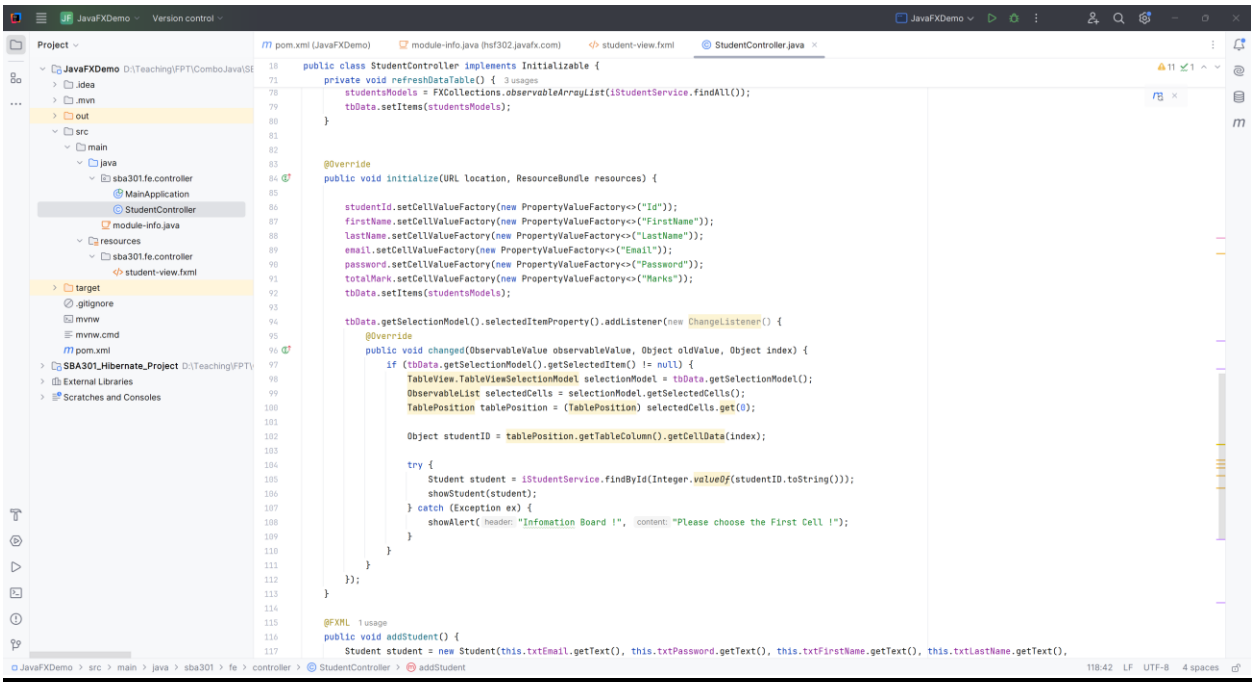
    @FXML 1 usage
    public void deleteStudent() {
        iStudentService.delete(this.getIdStudent());
        refreshDataTable();
    }

    @FXML 1 usage
    public void updateStudent() {
        Student student = new Student(this.idStudent, this.txtEmail.getText(), this.txtPassword.getText(),
                this.txtFirstName.getText(),
                this.txtLastName.getText(),
                Integer.parseInt(txtTotalMark.getText()));
        iStudentService.update(student);
        refreshDataTable();
    }

    public int getIdStudent() { return idStudent; }

    public void setIdStudent(int idStudent) { this.idStudent = idStudent; }
}
```

Step 09. Show all student and get StudentID when click first cell



```
public class StudentController implements Initializable {
    private void refreshDataTable() { 3 usages
        studentsModels = FXCollections.observableArrayList(iStudentService.findAll());
        tbData.setItems(studentsModels);
    }


    @Override
    public void initialize(URL location, ResourceBundle resources) {

        studentId.setCellValueFactory(new PropertyValueFactory<>("Id"));
        firstName.setCellValueFactory(new PropertyValueFactory<>("FirstName"));
        lastName.setCellValueFactory(new PropertyValueFactory<>("LastName"));
        email.setCellValueFactory(new PropertyValueFactory<>("Email"));
        password.setCellValueFactory(new PropertyValueFactory<>("Password"));
        totalMark.setCellValueFactory(new PropertyValueFactory<>("Marks"));
        tbData.setItems(studentsModels);

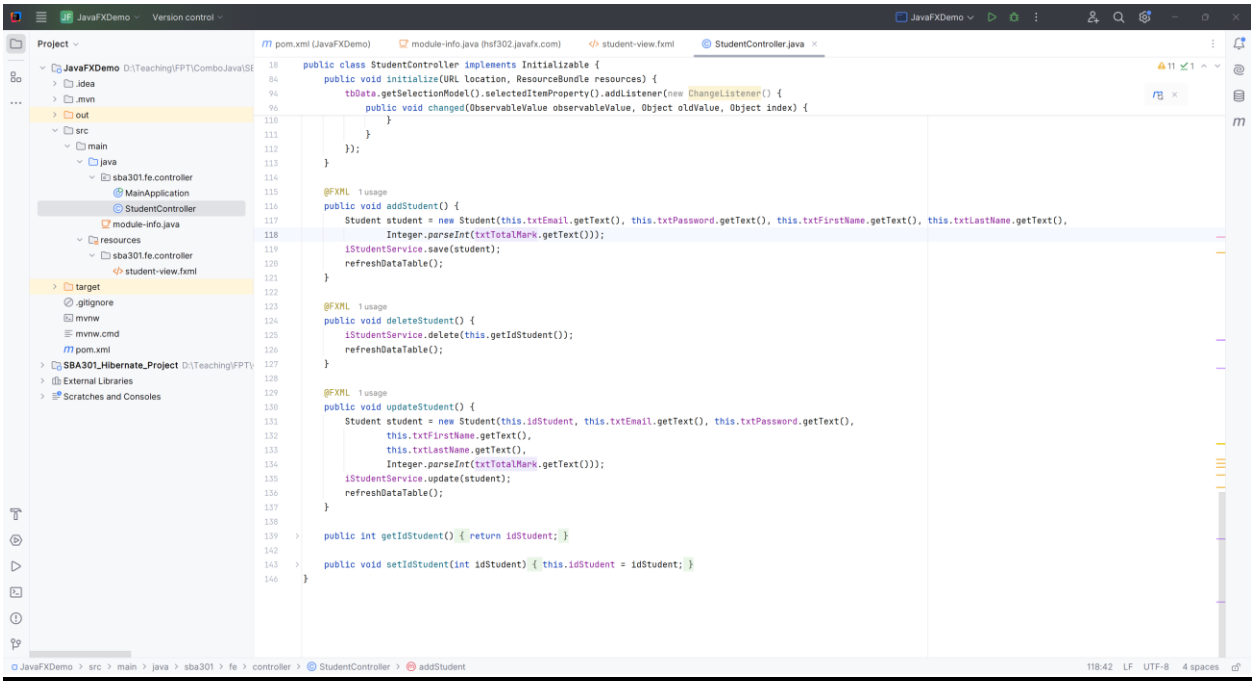
        tbData.getSelectionModel().selectedItemProperty().addListener(new ChangeListener() {
            @Override
            public void changed(ObservableValue observableValue, Object oldValue, Object index) {
                if (tbData.getSelectionModel().getSelectedItem() != null) {
                    TableView.TableViewSelectionModel selectionModel = tbData.getSelectionModel();
                    ObservableList selectedCells = selectionModel.getSelectedCells();
                    TablePosition tablePosition = (TablePosition) selectedCells.get(0);

                    Object studentID = tablePosition.getTableColumn().getCellData(index);

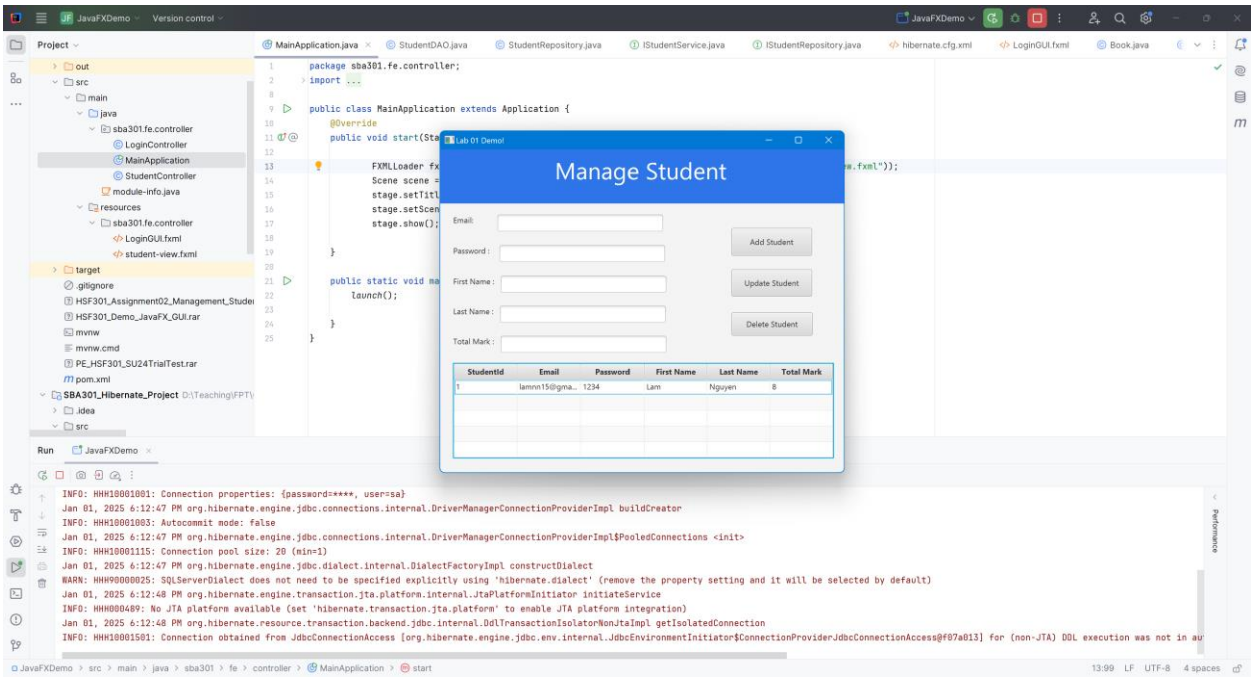
                    try {
                        Student student = iStudentService.findById(Integer.valueOf(studentID.toString()));
                        showStudent(student);
                    } catch (Exception ex) {
                        showAlert( header: "Infomation Board !",  content: "Please choose the First Cell !");
                    }
                }
            }
        });
    }

    @FXML 1 usage
    public void addStudent() {
        Student student = new Student(this.txtEmail.getText(), this.txtPassword.getText(), this.txtFirstName.getText(), this.txtLastName.getText(),
```

Step 10. CRUD for Students

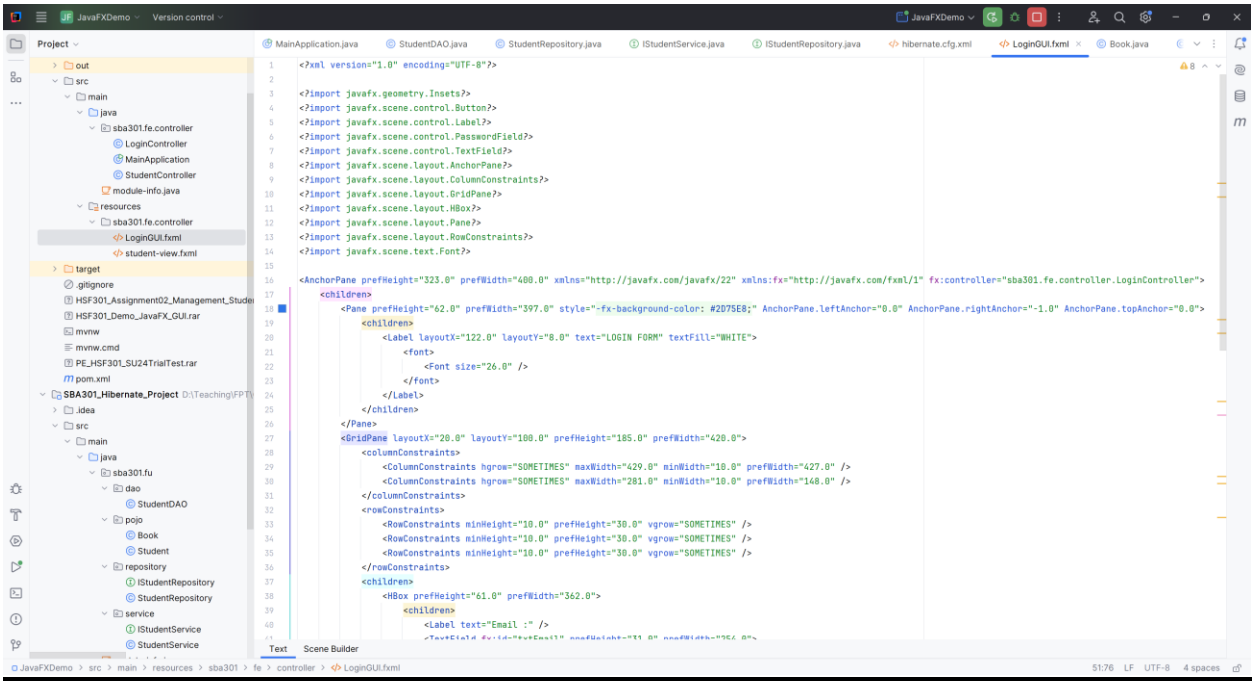


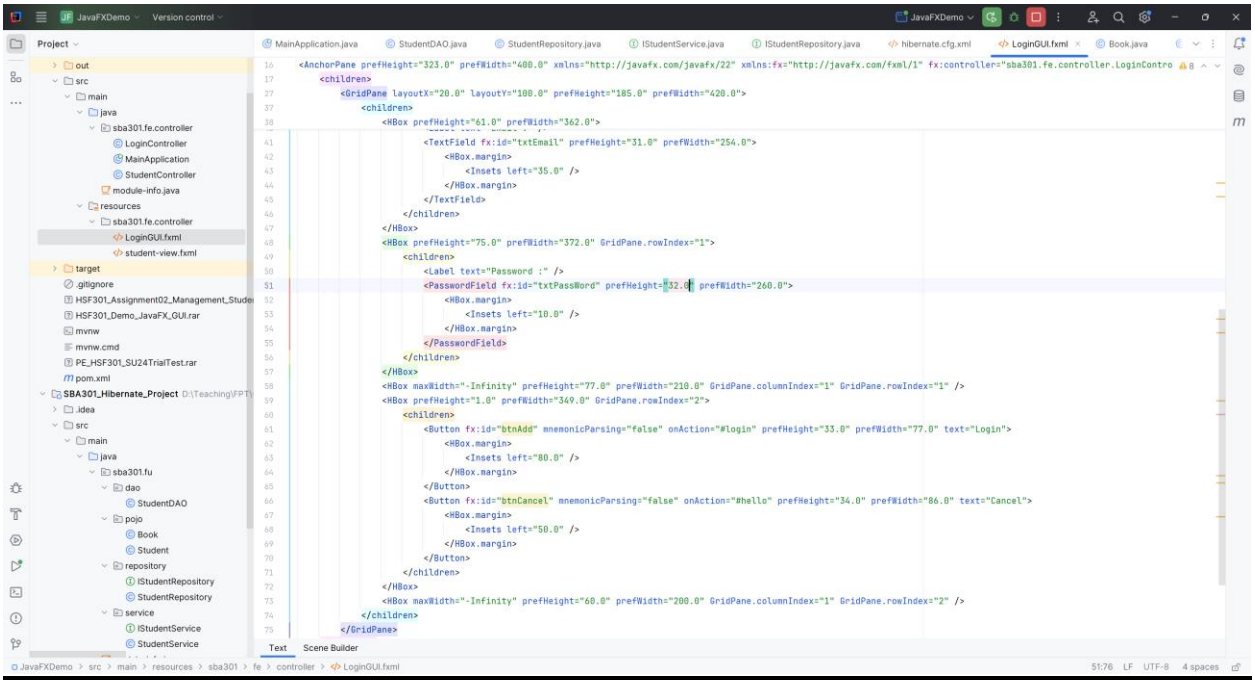
Step 11. Run Project

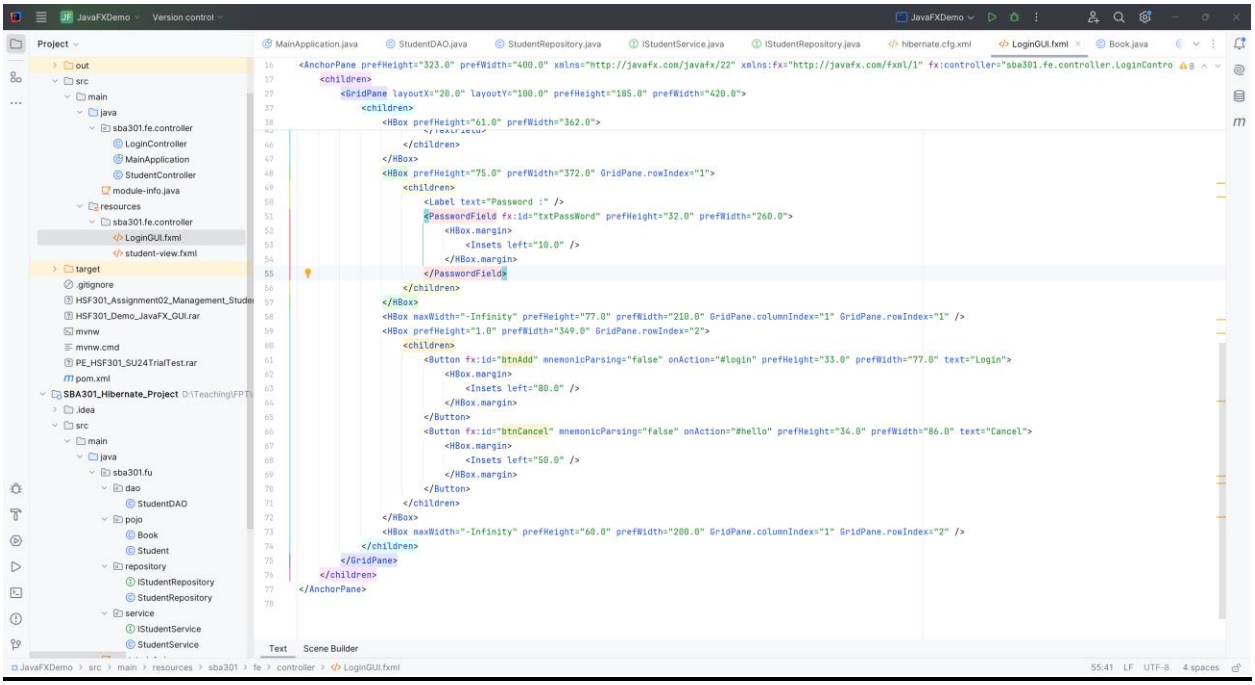


Activity 04: Design and Develop the Login Page

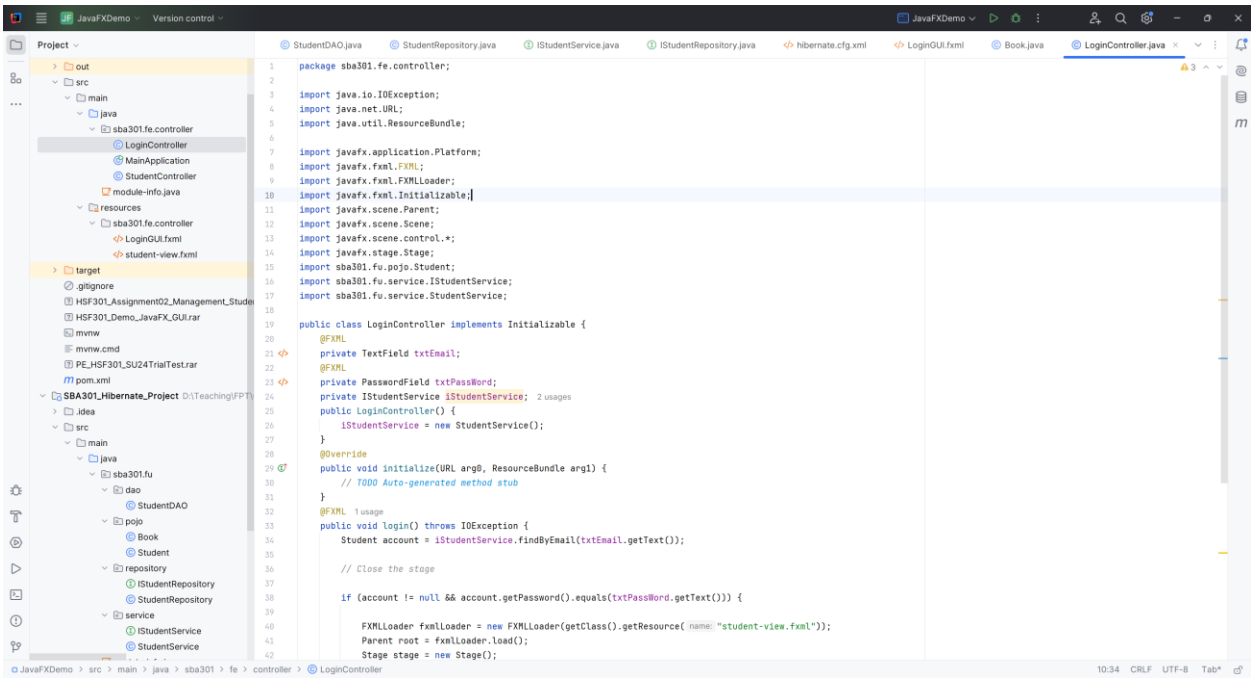
Step 01. Design the LoginGUI.fxml

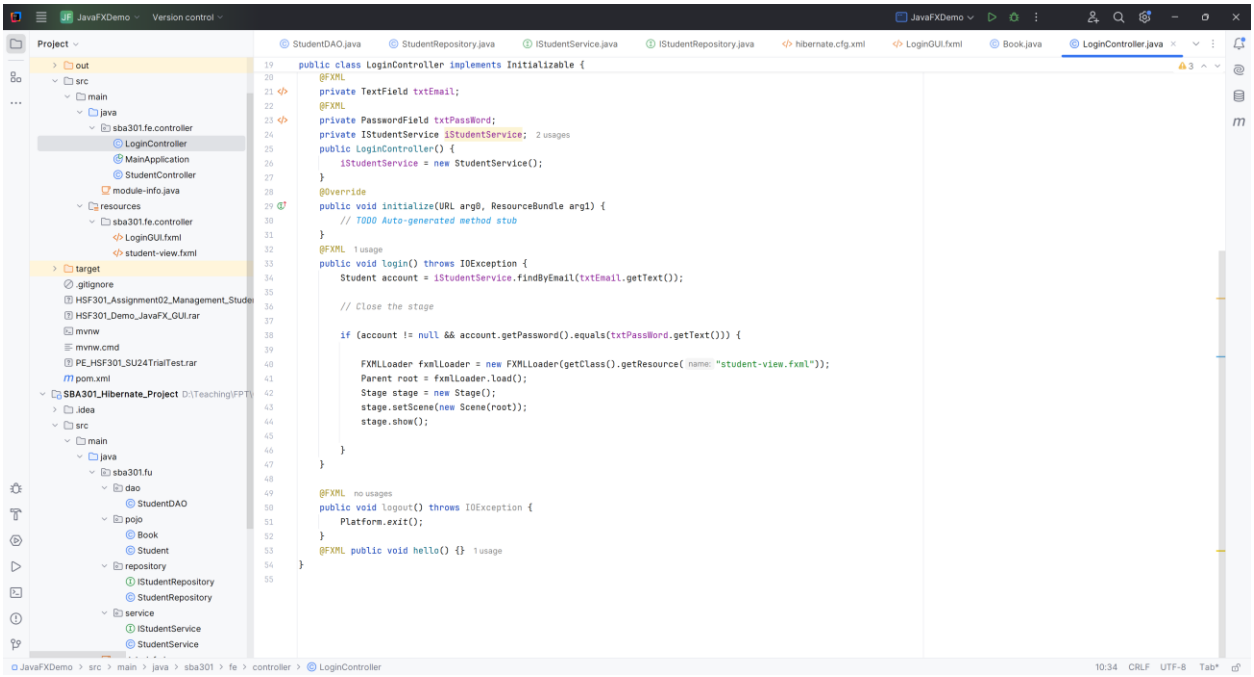




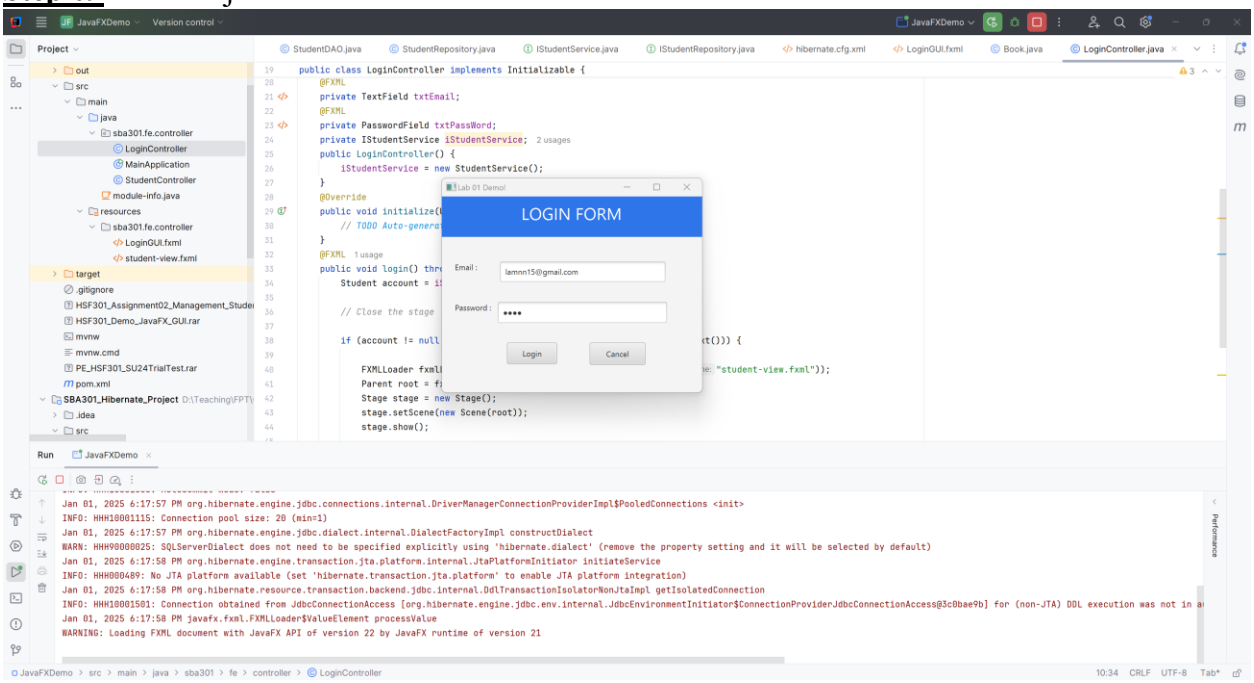


Step 02. Create LoginController.java in Controller Package





Step 09. Run Project



After Login Success

